

TRƯỜNG ĐẠI HỌC FPT

CAPSTONE PROJECT REPORT

Report 4 – Software Design Document

– Ho Chi Minh, October 25 –

Table of Contents

I. Record of Changes..... 3

II. Software Design Document..... 4

1. System Design..... 4

1.1 System Architecture..... 4

1.2 Package Diagram..... 4

2. Database Design..... 4

3. Detailed Design..... 5

3.1 <Feature/Function Name1>..... 5

3.2 <Feature/Function Name2>..... 6

I. Record of Changes

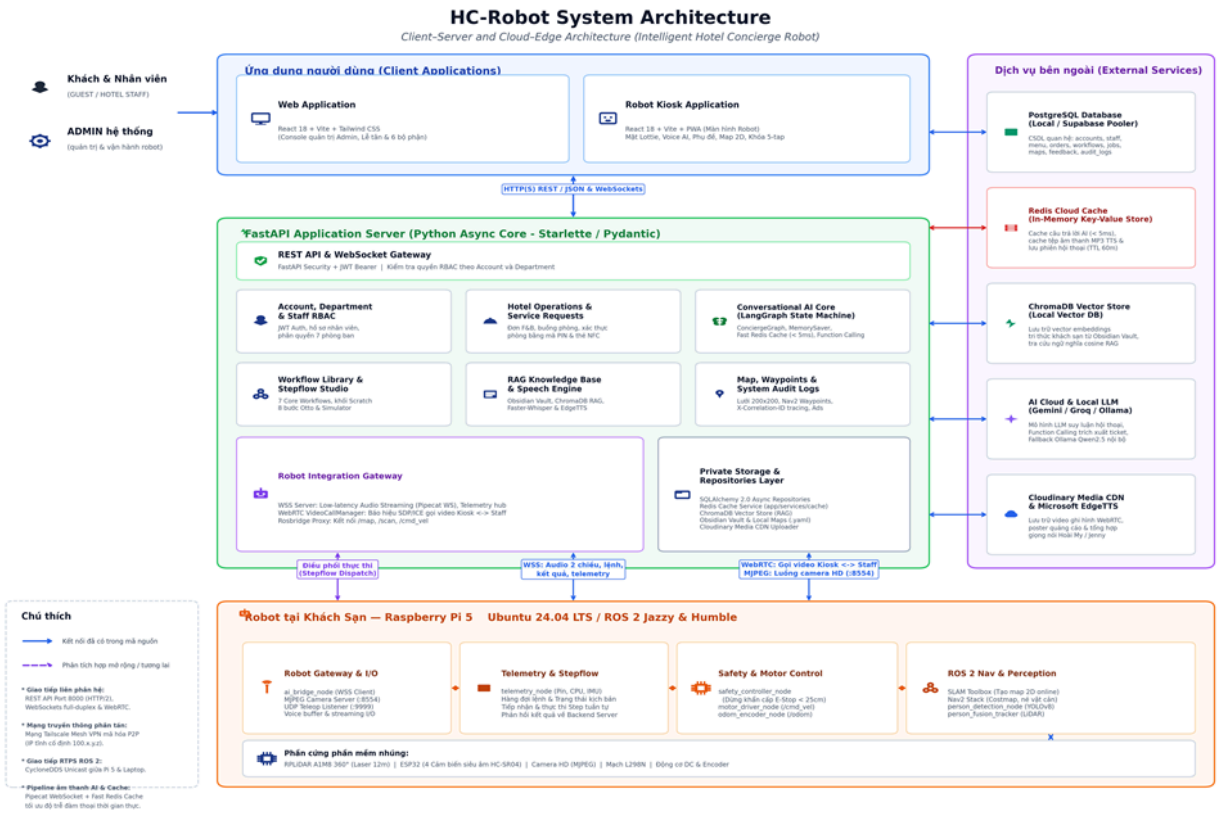
Date	A* M, D	In charge	Change Description

*A - Added M - Modified D - Deleted

II. Software Design Document

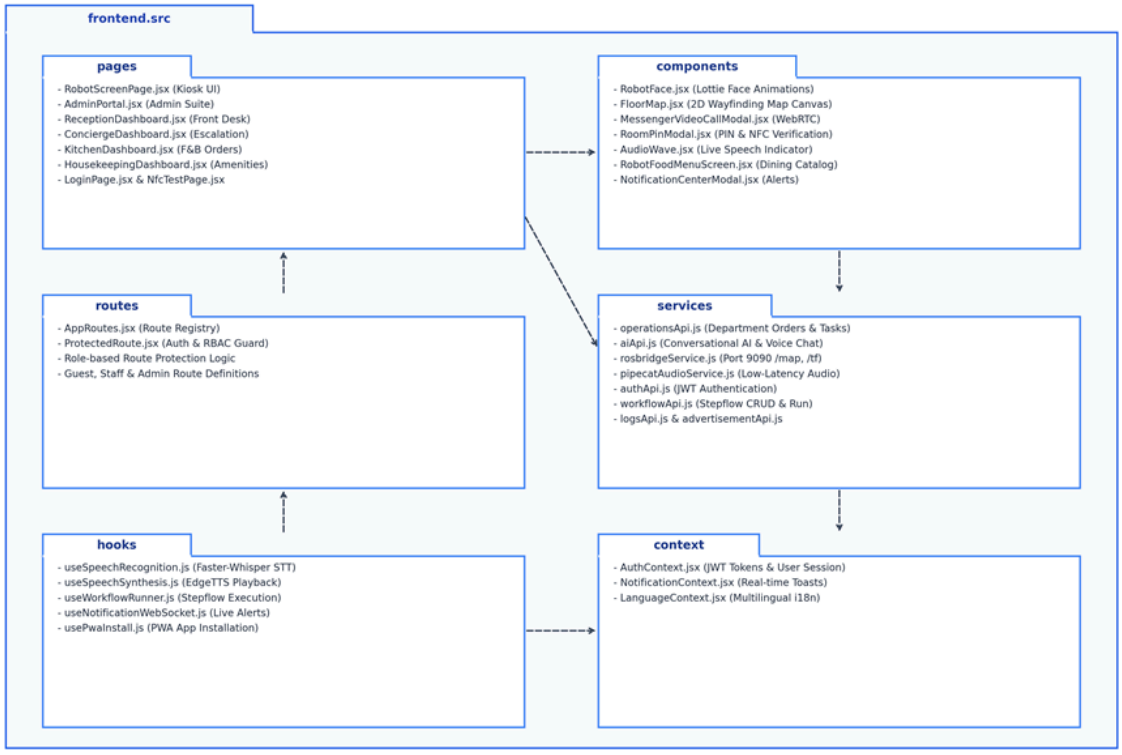
1. System Design

1.1 System Architecture



1.2 Package Diagram

1.2.1 Frontend Web & Kiosk Console Package Diagram

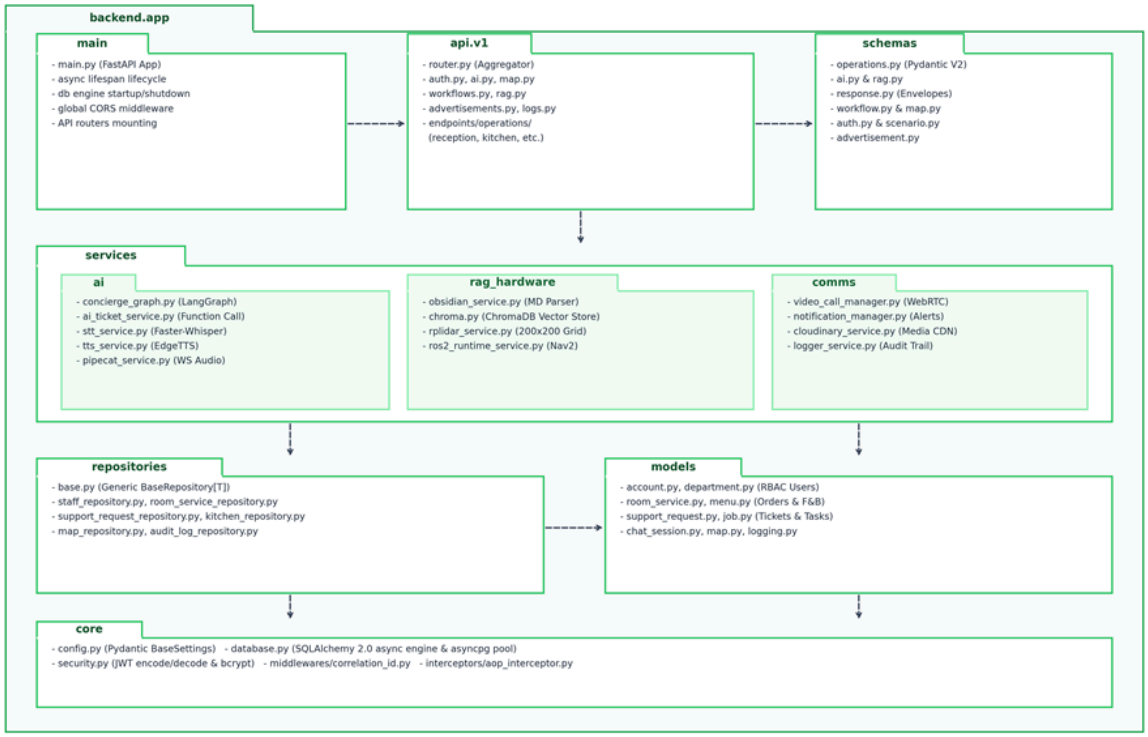


Package Descriptions

No	Package	Description
01	**frontend.src.pages**	Chứa các trang giao diện người dùng chính: Kiosk robot tương tác khách hàng (RobotScreenPage.jsx), bảng điều khiển quản trị (AdminPortal.jsx), đăng nhập (LoginPage.jsx), kiểm thử NFC (NfcTestPage.jsx), và 6 dashboard vận hành phòng ban (Reception, Concierge, Kitchen, Housekeeping, Bell Services, Maintenance).
02	**frontend.src.components**	Chứa các UI component tái sử dụng: RobotFace.jsx (Lottie face tương tác), FloorMap.jsx (bản đồ số 2D điều hướng wayfinding), MessengerVideoCallModal.jsx (gọi video call WebRTC với lễ tân), RoomPinModal.jsx (xác thực mã PIN phòng & thẻ NFC), AudioWave.jsx, và RobotFoodMenuScreen.jsx.
03	**frontend.src.routes**	Khai báo và cấu hình hệ thống điều hướng client-side: AppRoutes.jsx (đăng ký các tuyến đường URL) và ProtectedRoute.jsx (bảo vệ các route theo phiên xác thực và phân quyền RBAC theo vai trò nhân viên).
04	**frontend.src.services**	Đóng gói các hàm và API client kết nối backend qua Axios và WebSockets: operationsApi.js (quản lý đơn hàng và tác vụ phòng ban), aiApi.js (hội thoại AI), rosbridgeService.js (kết nối cổng 9090 nhận dữ liệu /map và /tf), pipecatAudioService.js (audio WebSocket), và authApi.js.

05	**frontend.src.hooks**	Chứa các custom React hooks đóng gói logic trạng thái: useSpeechRecognition.js (Faster-Whisper STT), useSpeechSynthesis.js (EdgeTTS phát giọng nói), useWorkflowRunner.js (thực thi kịch bản nhiệm vụ robot), useNotificationWebSocket.js, và usePwaInstall.js.
06	**frontend.src.context**	Quản lý trạng thái toàn cục của ứng dụng qua React Context: AuthContext.jsx (quản lý token JWT và phiên làm việc người dùng), NotificationContext.jsx (quản lý thông báo nổi realtime), và LanguageContext.jsx (hỗ trợ đa ngôn ngữ tiếng Việt và tiếng Anh).

1.2.2 Central Backend Server Package Diagram

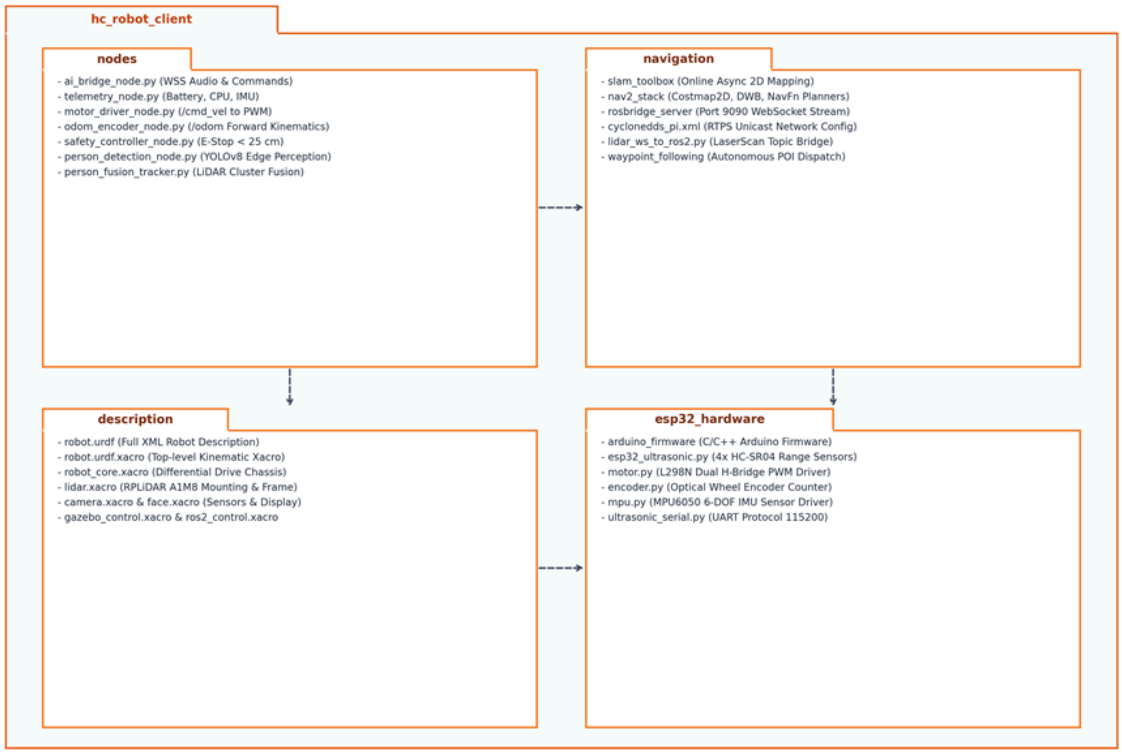


Package Descriptions

No	Package	Description
01	**backend.app.main**	Điểm khởi tạo ứng dụng FastAPI: cấu hình vòng đời bất đồng bộ lifespan (khởi tạo engine CSDL, kiểm tra pool kết nối, đóng tài nguyên khi tắt), đăng ký cấu hình CORS toàn cục và gắn kết các router API phiên bản /api/v1.
02	**backend.app.api.v1**	Tầng định tuyến API: router.py đóng gói và phân phối request đến các router thành phần (auth.py, ai.py, map.py, workflows.py, rag.py, advertisements.py, logs.py) và 7 router nghiệp vụ phòng ban (reception, kitchen, room_service, housekeeping, concierge...).

03	**backend.app.services.ai**	Xử lý hội thoại AI và tiếng nói: concierge_graph.py (LangGraph state machine với MemorySaver), ai_ticket_service.py (trích xuất ticket đơn hàng từ lời nói qua Function Calling), stt_service.py (Faster-Whisper), tts_service.py (EdgeTTS), và pipecat_service.py.
04	**backend.app.services.rag**	Động cơ tri thức RAG: obsidian_service.py (quét, đọc và phân đoạn tài liệu Markdown từ Obsidian Vault) và chroma.py (quản lý CSDL vector ChromaDB, tra cứu ngữ nghĩa cosine similarity để cung cấp tri thức cho LLM).
05	**backend.app.services.hardware**	Tích hợp phần cứng và điều phối robot: rplidar_service.py (chuyển đổi luồng quét laser RPLiDAR thành lưới bản đồ chiếm chỗ 200x200 gồm 40.000 ô và xuất tệp bản đồ .png/.json) và ros2_runtime_service.py (gửi mục tiêu điều hướng Nav2).
06	**backend.app.services**	Các dịch vụ truyền thông và media dùng chung: video_call_manager.py (quản lý phòng báo hiệu WebRTC SDP/ICE kết nối cuộc gọi giữa robot và nhân viên), notification_manager.py (phát thông báo đẩy qua WebSockets), và cloudinary_service.py (lưu trữ video cuộc gọi trên CDN).
07	**backend.app.repositories**	Tầng truy cập dữ liệu (Repository Pattern): khai báo lớp cơ sở base.py (BaseRepository[T]) và các repository nghiệp vụ (StaffRepository, RoomServiceRepository, KitchenRepository, SupportRequestRepository, MapRepository, AuditLogRepository) phân tách truy vấn CSDL.
08	**backend.app.models**	Định nghĩa các ORM models SQLAlchemy ánh xạ các bảng CSDL PostgreSQL: account.py, department.py (tài khoản & phân quyền), room_service.py, menu.py (đặt món ăn), support_request.py, job.py (ticket & nhiệm vụ robot), chat_session.py, map.py, và logging.py.
09	**backend.app.schemas**	Định nghĩa cấu trúc dữ liệu Pydantic V2: operations.py (dữ liệu đơn hàng, phòng ban), ai.py (schema hội thoại), rag.py, response.py (chuẩn hóa envelope phản hồi API), workflow.py (cấu trúc kịch bản khối Scratch), map.py, và auth.py.
10	**backend.app.core**	Các thành phần hạ tầng dùng chung: config.py (Pydantic BaseSettings đọc biến môi trường), database.py (khởi tạo async engine và sessionmaker với asyncpg), security.py (mã hóa mật khẩu bcrypt và JWT), middlewares/correlation_id.py, và interceptors/aop_interceptor.py.

1.2.3 Robot Edge Computing Package Diagram



Package Descriptions

No	Package	Description
01	**hc_robot_client.nodes**	Chứa toàn bộ các ROS 2 computational nodes thực thi trên Raspberry Pi 5: ai_bridge_node.py (cầu nối WebSocket hai chiều truyền âm thanh và nhận lệnh điều khiển), telemetry_node.py (thu thập và gửi thông số pin, CPU, IMU), motor_driver_node.py (/cmd_vel to PWM), odom_encoder_node.py (tính toán động học vi sai xuất ra /odom), safety_controller_node.py (dừng khẩn cấp E-Stop khi gặp vật cản < 25 cm), person_detection_node.py (nhận diện người bằng YOLOv8), và person_fusion_tracker.py (bám theo khách).
02	**robot.navigation**	Chứa các tệp cấu hình và mã điều hướng tự hành: slam_toolbox (thuật toán Online Async 2D SLAM quét tạo bản đồ /map), nav2_stack (bộ điều hướng Nav2 gồm costmap_2d, NavFn planner, DWB controller), rosbridge_server (máy chủ WebSocket cổng 9090 phát trực tiếp dữ liệu /map, /scan, /tf lên web), cyclonedds_pi.xml (cấu hình mạng Unicast DDS qua Tailscale), và lidar_ws_to_ros2.py.
03	**robot.description**	Chứa các tệp mô tả hình học và động học 3D của robot: robot.urdf và robot.urdf.xacro (mô tả tổng thể cây liên kết link và khớp joint), robot_core.xacro (khung gầm xe, bánh dẫn động, bánh caster), lidar.xacro (vị trí lắp đặt cảm biến RPLiDAR A1M8), camera.xacro và face.xacro (mô tả camera HD và màn hình hiển thị), và gazebo_control.xacro.

04	**robot.esp32_hardware**	Chứa firmware nhúng và driver điều khiển phần cứng cấp thấp trên ESP32: arduino_firmware (mã nguồn C/C++ nạp trực tiếp cho vi điều khiển), esp32_ultrasonic.py (đọc dữ liệu 4 cảm biến siêu âm HC-SR04 đặt tại 4 góc robot), motor.py (điều khiển mạch cầu H kép L298N cấp xung cho 2 động cơ DC), encoder.py (đo tốc độ quay bánh xe), mpu.py (cảm biến quán tính IMU 6 trục), và ultrasonic_serial.py (UART Serial tốc độ 115200 baud).

2. Database Design

2.1 Operations Subsystem

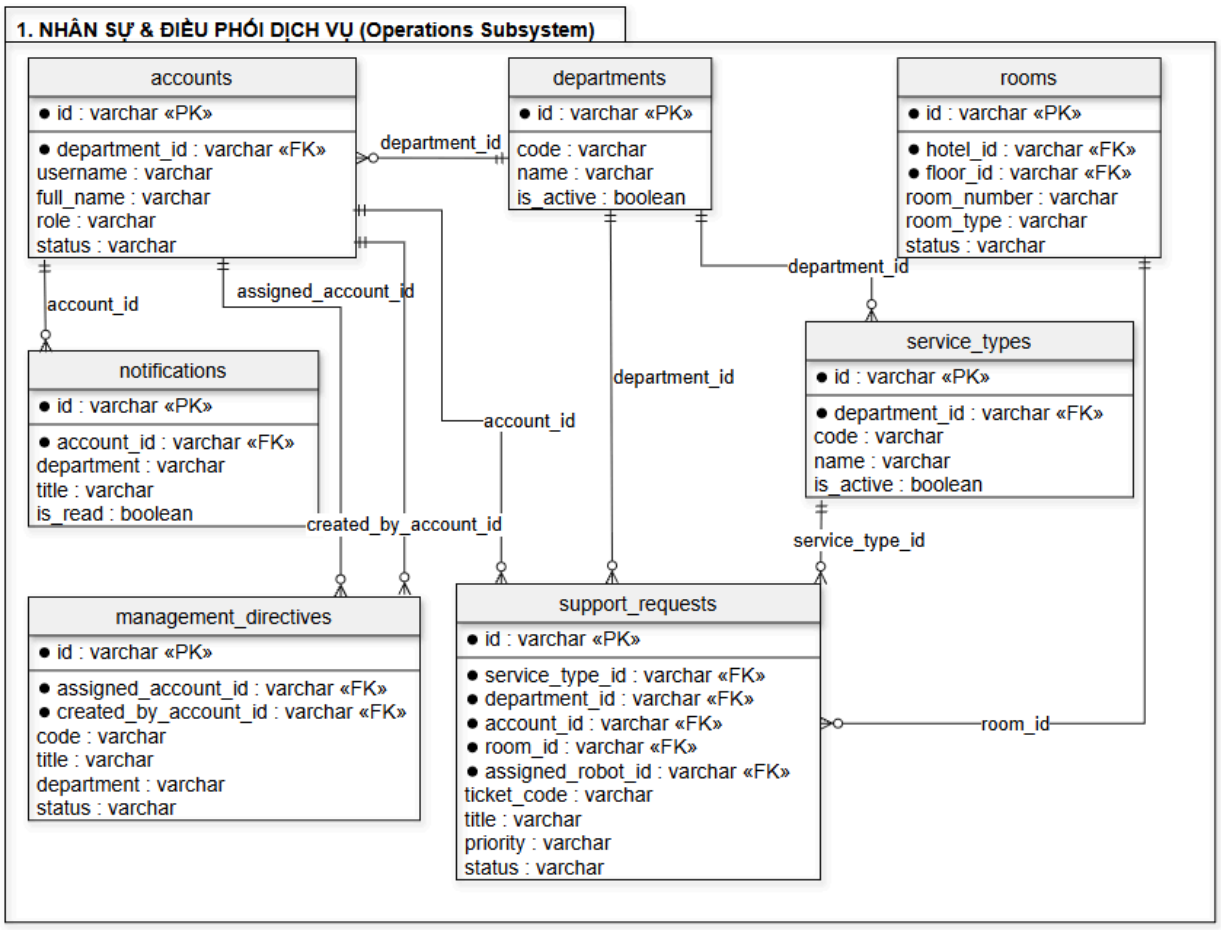


Table Descriptions

No	Table	Description
1	accounts	Stores staff accounts, department membership, roles, and account status. - Primary keys: id

		- Foreign keys: department_id
2	departments	Defines hotel departments responsible for handling services and coordinating staff. - Primary keys: id - Foreign keys: None
3	rooms	Stores hotel room details and availability/status for service delivery. - Primary keys: id - Foreign keys: hotel_id, floor_id
4	notifications	Stores notifications addressed to staff accounts, including their title, department, and read status. - Primary keys: id - Foreign keys: account_id
5	service_types	Defines service categories and the department responsible for each service type. - Primary keys: id - Foreign keys: department_id
6	management_directives	Stores management instructions, their creator, assigned staff member, department, and processing status. - Primary keys: id - Foreign keys: assigned_account_id, created_by_account_id
7	support_requests	Stores service tickets with their service type, responsible department, associated account and room, assigned robot, priority, and status. - Primary keys: id - Foreign keys: service_type_id, department_id, account_id, room_id, assigned_robot_id

2.2 F&B Subsystem

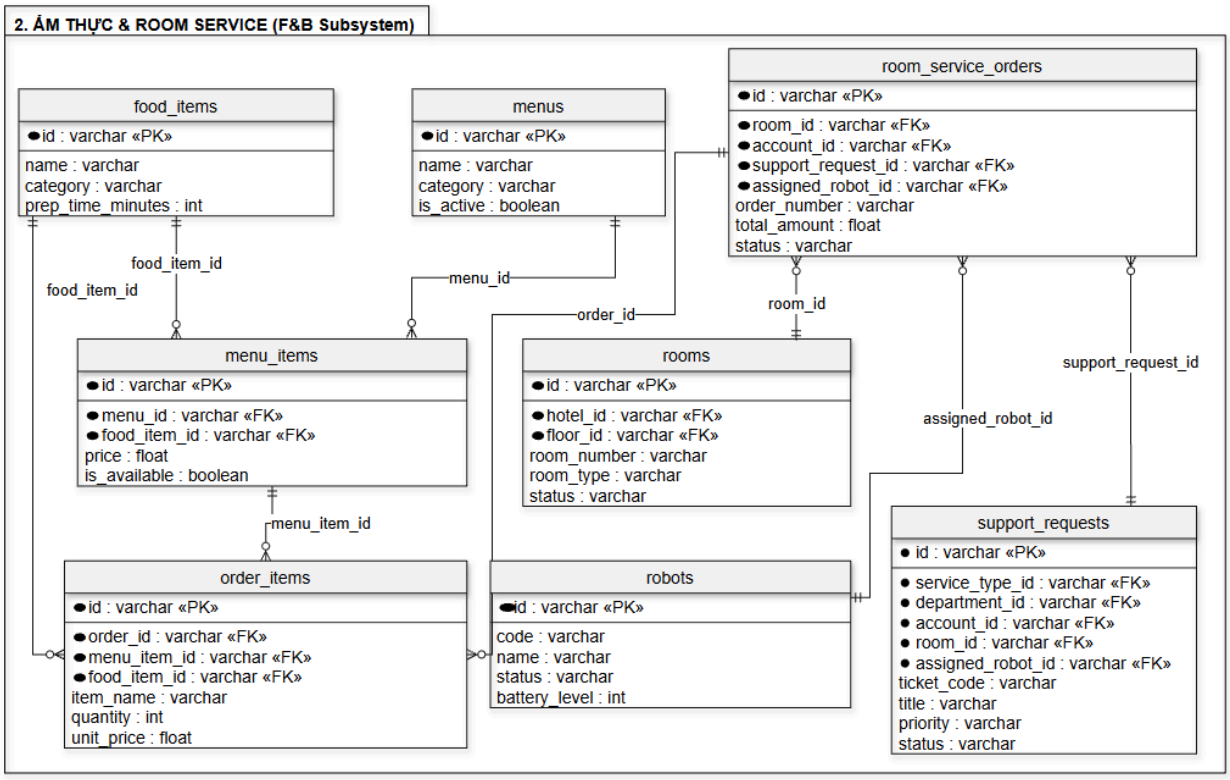


Table Descriptions

No	Table	Description
1	food_items	Stores the food catalog, including item names, categories, and preparation times. - Primary keys: id - Foreign keys: None
2	menus	Defines menus and their categories and active status. - Primary keys: id - Foreign keys: None
3	menu_items	Associates food items with menus and stores menu-specific prices and availability. - Primary keys: id - Foreign keys: menu_id, food_item_id
4	room_service_orders	Stores room-service orders, their ordering account and destination room, linked support request, assigned robot, total amount, and status. - Primary keys: id - Foreign keys: room_id, account_id, support_request_id, assigned_robot_id
5	order_items	Stores individual items in each order, including menu and food references, item name, quantity, and unit price. - Primary keys: id

		- Foreign keys: order_id, menu_item_id, food_item_id
6	rooms	Stores the hotel rooms used as room-service delivery destinations. - Primary keys: id - Foreign keys: hotel_id, floor_id
7	robots	Stores robot identities, operational status, and battery levels for delivery assignment. - Primary keys: id - Foreign keys: None
8	support_requests	Stores service tickets linked to room-service orders and tracks their department, priority, status, and robot assignment. - Primary keys: id - Foreign keys: service_type_id, department_id, account_id, room_id, assigned_robot_id

2.3 AI Experience & Logs

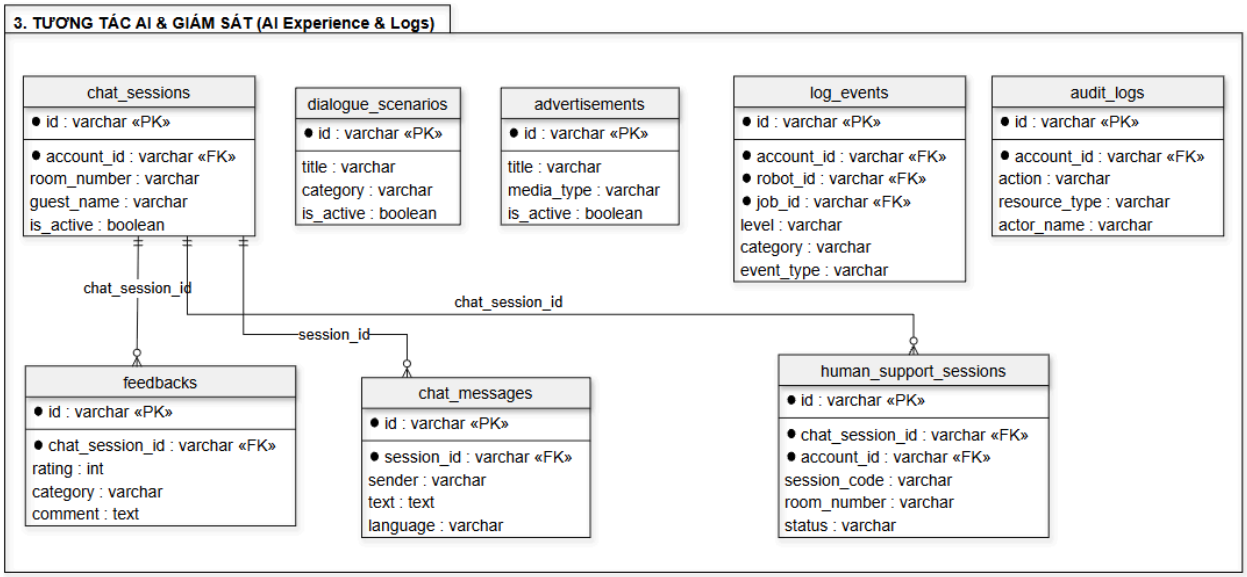


Table Descriptions

No	Table	Description
1	chat_sessions	Stores guest conversation sessions with an associated account, room number, guest name, and active status. - Primary keys: id - Foreign keys: account_id
2	dialogue_scenarios	Defines dialogue scenario titles and categories available to the AI interaction service. - Primary keys: id - Foreign keys: None
3	advertisements	Stores advertisement titles, media types, and active status for content presentation.

		- Primary keys: id - Foreign keys: None
4	chat_messages	Stores conversation messages, including the sender, message text, and language within a chat session. - Primary keys: id - Foreign keys: session_id
5	feedbacks	Stores guest ratings, feedback categories, and comments associated with a chat session. - Primary keys: id - Foreign keys: chat_session_id
6	human_support_sessions	Tracks human assistance sessions linked to AI conversations, including the supporting account, session code, room number, and status. - Primary keys: id - Foreign keys: chat_session_id, account_id
7	log_events	Records system and robot events with their severity level, category, event type, and associated account, robot, or job. - Primary keys: id - Foreign keys: account_id, robot_id, job_id
8	audit_logs	Records account actions and affected resource types for traceability and auditing. - Primary keys: id - Foreign keys: account_id

2.4 AI Experience & Logs

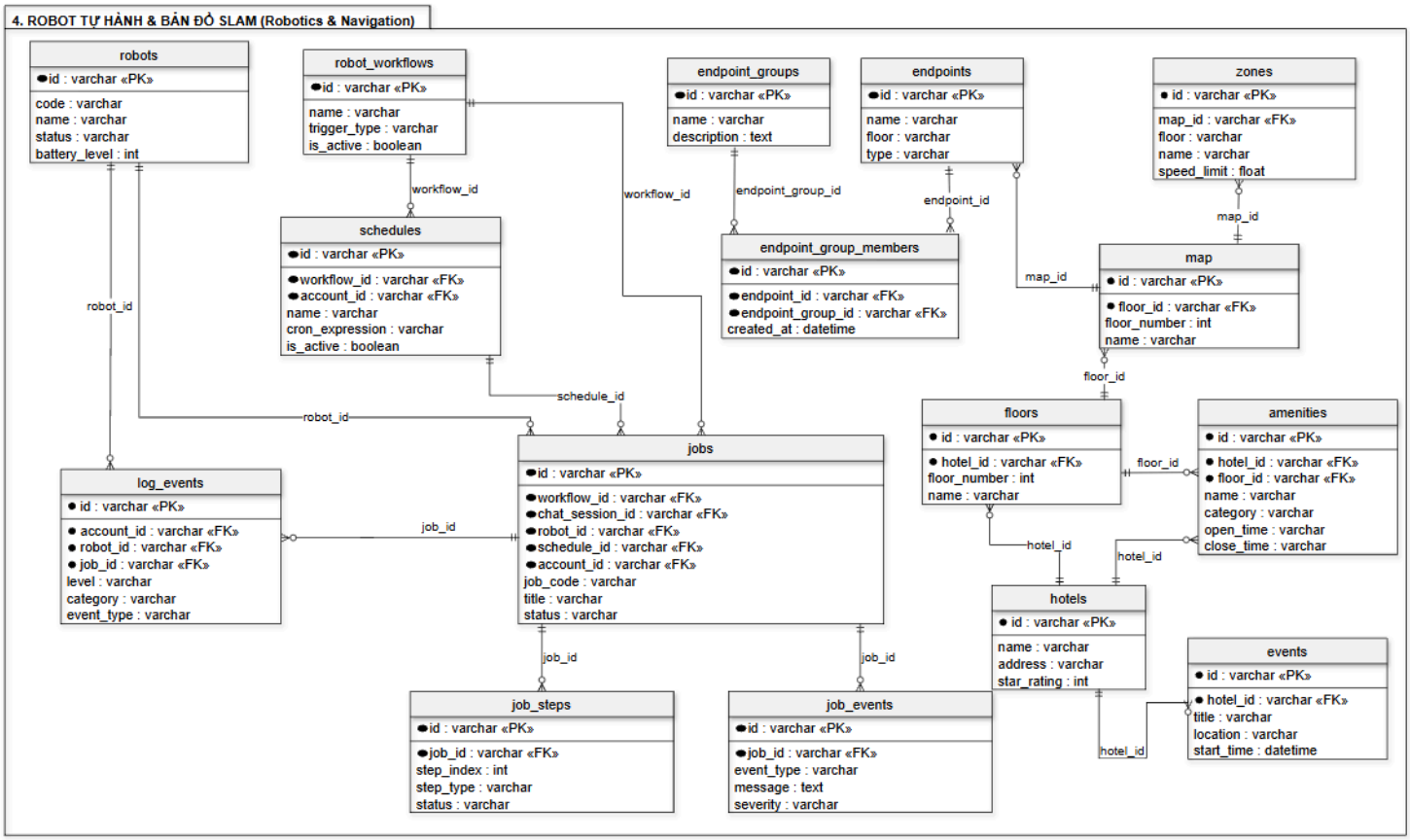


Table Descriptions

No	Table	Description
1	robots	Stores robot identifiers, codes, names, operational status, and battery levels. - Primary keys: id - Foreign keys: None
2	robot_workflows	Defines reusable robot workflows, their trigger types, and active status. - Primary keys: id - Foreign keys: None
3	schedules	Stores named recurring schedules, their cron expressions, associated workflows, creating/managing accounts, and active status. - Primary keys: id - Foreign keys: workflow_id, account_id
4	jobs	Stores robot task executions and links each job to its workflow, chat session, robot, schedule, and account while tracking its code, title, and status.

		- Primary keys: id - Foreign keys: workflow_id, chat_session_id, robot_id, schedule_id, account_id
5	job_steps	Stores the ordered execution steps of a robot job, including step index, type, and status. - Primary keys: id - Foreign keys: job_id
6	job_events	Records events generated during a job, including event type, message, and severity. - Primary keys: id - Foreign keys: job_id
7	log_events	Stores system and robot event logs associated with accounts, robots, and jobs for monitoring and troubleshooting. - Primary keys: id - Foreign keys: account_id, robot_id, job_id
8	endpoint_groups	Defines named groups of navigation endpoints with descriptive information. - Primary keys: id - Foreign keys: None
9	endpoints	Stores named navigation destinations with their floor and endpoint type. - Primary keys: id - Foreign keys: map_id
10	endpoint_group_members	Associates navigation endpoints with endpoint groups and records when each membership was created. - Primary keys: id - Foreign keys: endpoint_id, endpoint_group_id
11	map	Stores navigation maps associated with hotel floors, including the floor number and map name. - Primary keys: id - Foreign keys: floor_id
12	zones	Defines named navigation zones within a map and their floor and speed limit. - Primary keys: id - Foreign keys: map_id
13	floors	Stores hotel floor numbers and names for organizing maps, rooms, and amenities. - Primary keys: id - Foreign keys: hotel_id
14	hotels	Stores hotel names, addresses, and star ratings. - Primary keys: id - Foreign keys: None

15	amenities	Stores hotel amenities, their locations by floor, categories, and opening and closing times. - Primary keys: id - Foreign keys: hotel_id, floor_id
16	events	Stores hotel events, including their titles, locations, and start times. - Primary keys: id - Foreign keys: hotel_id

3. Detailed Design

This section describes the design supplied for code revision 99f3f97. Python modules are identified as modules in the class diagrams; they are not presented as instantiated classes. The diagrams show feature-relevant members, while the specification tables retain the supplied signatures. Report 3 references identify related requirements, not a claim that every requirement has been implemented. Hardware execution and external-service behavior require integration validation.

3.1 Staff Authentication and Profile

Staff submit credentials to AuthRouter. StaffRepository resolves the account and department; SecurityCore verifies the password and issues a JWT. The response supplies the user profile and departmental dashboard. Profile and password changes use the same authentication boundary.

Report 3: 3.2 Authentication & Profile; 3.1.3 Screen Authorization.

3.1.1 Class Diagram

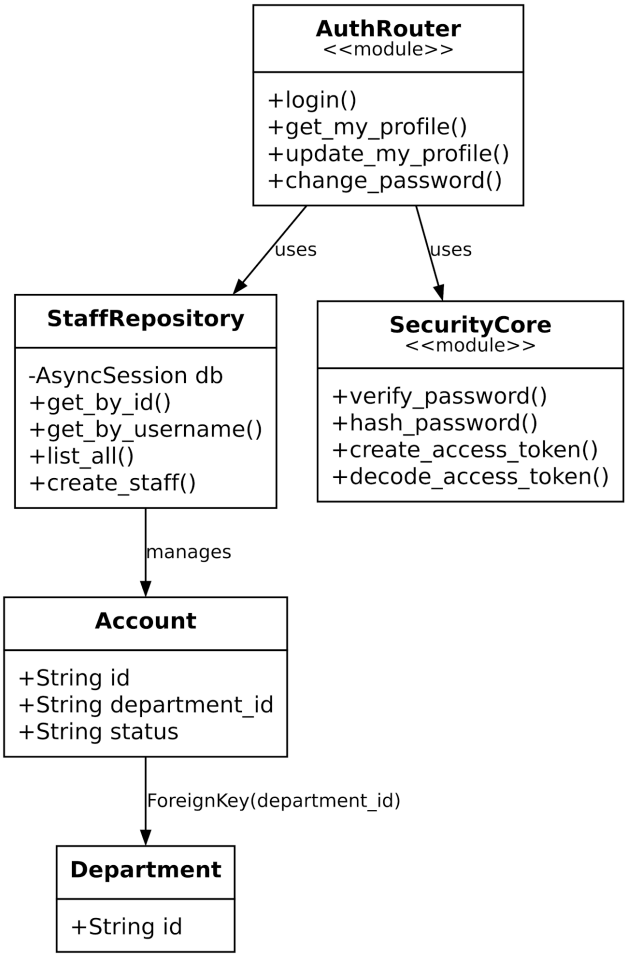


Figure 3.1-1. Staff Authentication and Profile: class and module relationships.

Class Specifications

No	Class / Module	Attribute / Method	Description
1	Account	id Mapped[str] (String 50, PK)	Primary key (ACC-XXXXXX) Source: app/models/account.py:17
2	Account	username Mapped[str] (String 50, Unique, Index)	Unique username for login Source: app/models/account.py:18
3	Account	password_hash Mapped[str] (String 255)	Stored password hash Source: app/models/account.py:19
4	Account	department_id Mapped[Optional[str]] (FK)	Reference to departments.id Source: app/models/account.py:25
5	Account	default_dashboard Mapped[str]	Operational dashboard routing key Source: app/models/account.py:27
6	StaffRepository	get_by_username async (username: str) -> Optional[Staff]	Fetches staff by username with department eager loaded Source: app/repositories/staff_repository.py:35
7	StaffRepository	change_password async (staff_id: str, new_hash: str) -> bool	Updates password_hash and updated_at Source: app/repositories/staff_repository.py:105
8	SecurityCore	verify_password (plain_password: str, hashed_password: str) -> bool	Verifies a supplied password against the stored hash Source: app/core/security.py:20
9	SecurityCore	create_access_token (data: dict, expires_delta: Optional[timedelta]) -> str	Encodes JWT with SECRET_KEY & ALGORITHM Source: app/core/security.py:42
10	AuthRouter	login async (payload: LoginRequest, repo: StaffRepository) -> TokenResponse	Authenticates and returns JWT + dashboard route Source: app/api/v1/endpoints/auth.py:98
11	AuthRouter	resolve_dashboard_for_staff (staff: Staff) -> str	Normalizes staff role/department to target UI dashboard Source: app/api/v1/endpoints/auth.py:66

3.1.2 Authenticate staff and return their departmental dashboard

Participants: Client = Frontend Client / Kiosk; AuthAPI = AuthRouter; Repo = StaffRepository; DB = PostgreSQL; Sec = SecurityCore; Audit = LoggerService.

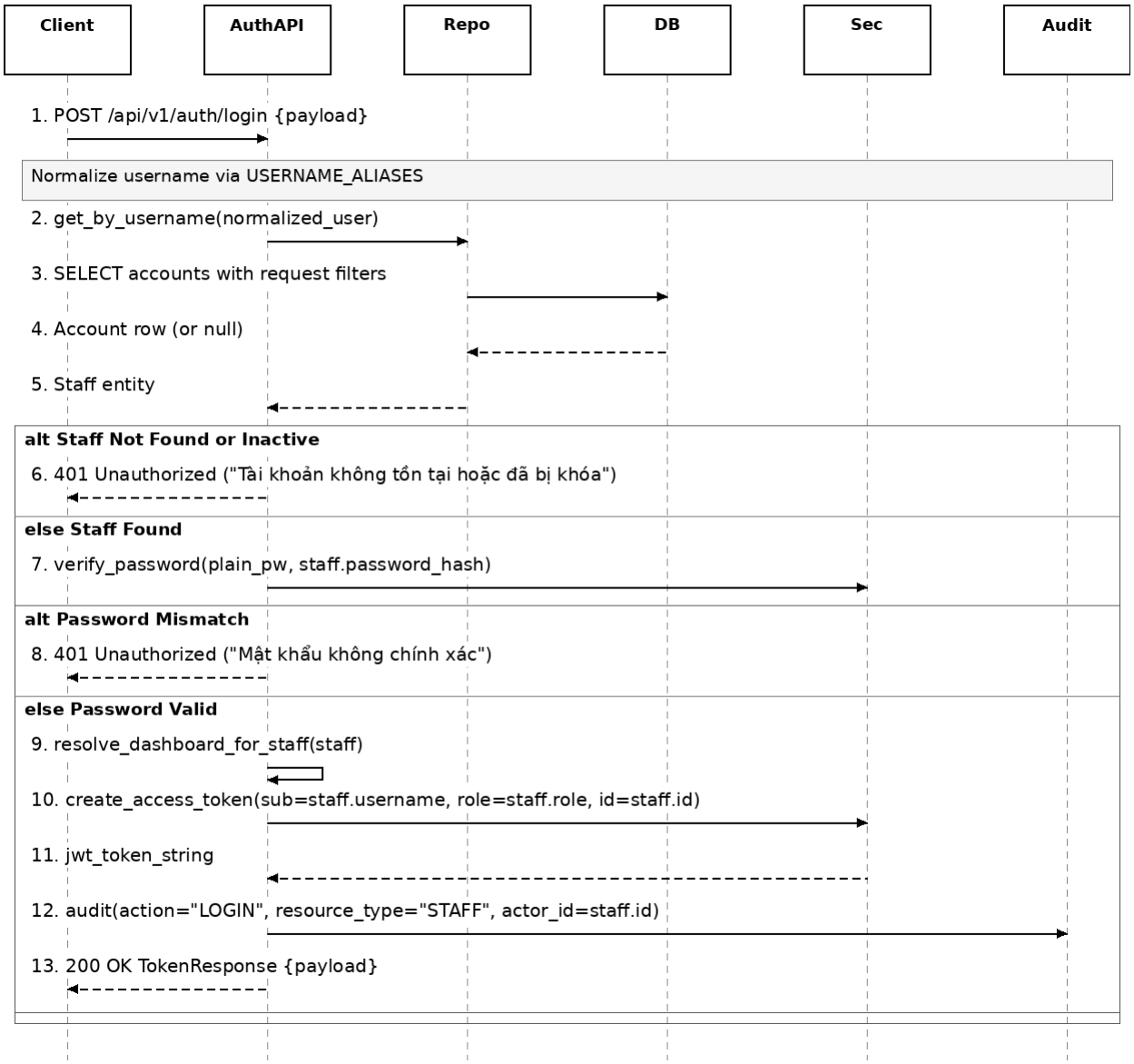


Figure 3.1-2. Authenticate staff and return their departmental dashboard.

Processing Rules and Implementation Limits

The login flow rejects absent/inactive accounts and invalid passwords with HTTP 401. Token validation is performed by get_current_user. Logout discards the client token; server-side revocation is not established by the handoff. Development-only password compatibility reported in the handoff must not be treated as the production authentication design.

3.2 Multilingual AI Voice Concierge

The guest submits speech or text through the kiosk. AIChatService orchestrates the concierge graph, optional speech synthesis and conversation persistence. ConciergeGraph checks cached DialogueScenario rules before calling a configured LLM. SessionManager retains bounded dialogue history and the response includes language, intent action and robot expression information.

Report 3: 3.3.2 Voice and Touch Conversation; 3.4.1 Speech Processing; 3.4.2 Answer Generation; 3.4.4 Conversation Session Management.

3.2.1 Class Diagram

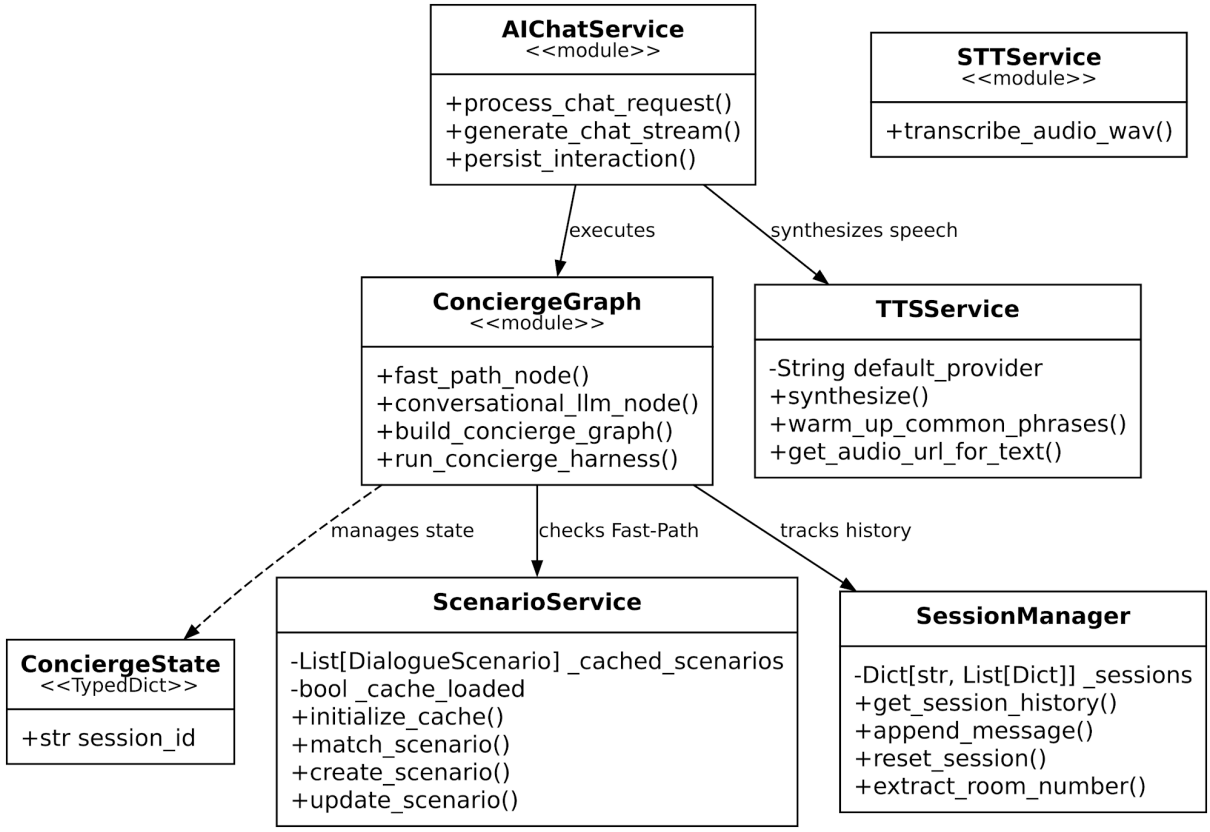


Figure 3.2-1. Multilingual AI Voice Concierge: class and module relationships.

Class Specifications

No	Class / Module	Attribute / Method	Description
1	ConciergeGraph	build_concierge_graph () -> CompiledStateGraph	Assembles LangGraph state machine with MemorySaver Source: app/services/ai/concierge_graph.py:348
2	ConciergeGraph	run_concierge_harness async (prompt: str, session_id: str, language: Optional[str], emotion: Optional[str]) -> Dict[str, Any]	Executes state machine from START to END Source: app/services/ai/concierge_graph.py:381
3	ConciergeGraph	fast_path_node async (state: ConciergeState) -> Dict[str, Any]	Node 1: Matches prompt against in-memory scenarios Source: app/services/ai/concierge_graph.py:52
4	ConciergeGraph	conversational_llm_node async (state: ConciergeState) -> Dict[str, Any]	Node 2: Invokes Ollama/Gemini with multi-turn history Source: app/services/ai/concierge_graph.py:118
5	ScenarioService	match_scenario (prompt: str, language: Optional[str]) -> Optional[Tuple]	Keyword/Regex scenario matcher against RAM cache Source: app/services/ai/scenario_service.py:126
6	SessionManager	append_message (session_id: str, role: str, content: str) -> None	Appends turn to bounded memory queue (max 10 turns) Source: app/services/ai/session_manager.py:46
7	TTSService	synthesize async (text: str, voice: Optional[str], language: str) -> bytes	Generates speech audio bytes via Edge-TTS / ElevenLabs Source: app/services/ai/tts_service.py:112
8	STTService	transcribe_audio_wav (audio_bytes: bytes, language: str) -> str	Converts WAV PCM audio to text Source: app/services/ai/stt_service.py:32
9	AIChatService	process_chat_request async (db: AsyncSession, request: ChatRequest) -> ChatResponse	Orchestrates STT, Harness, TTS, and DB persistence Source: app/services/ai/ai_chat_service.py:48

3.2.2 Process a guest question and return a spoken response

Participants: Guest = Hotel Guest; API = AI Router; ChatService = AIChatService; Graph = ConciergeGraph; Scenarios = ScenarioService; LLM = OllamaService / GeminiService; TTS = TTSService; DB = PostgreSQL.

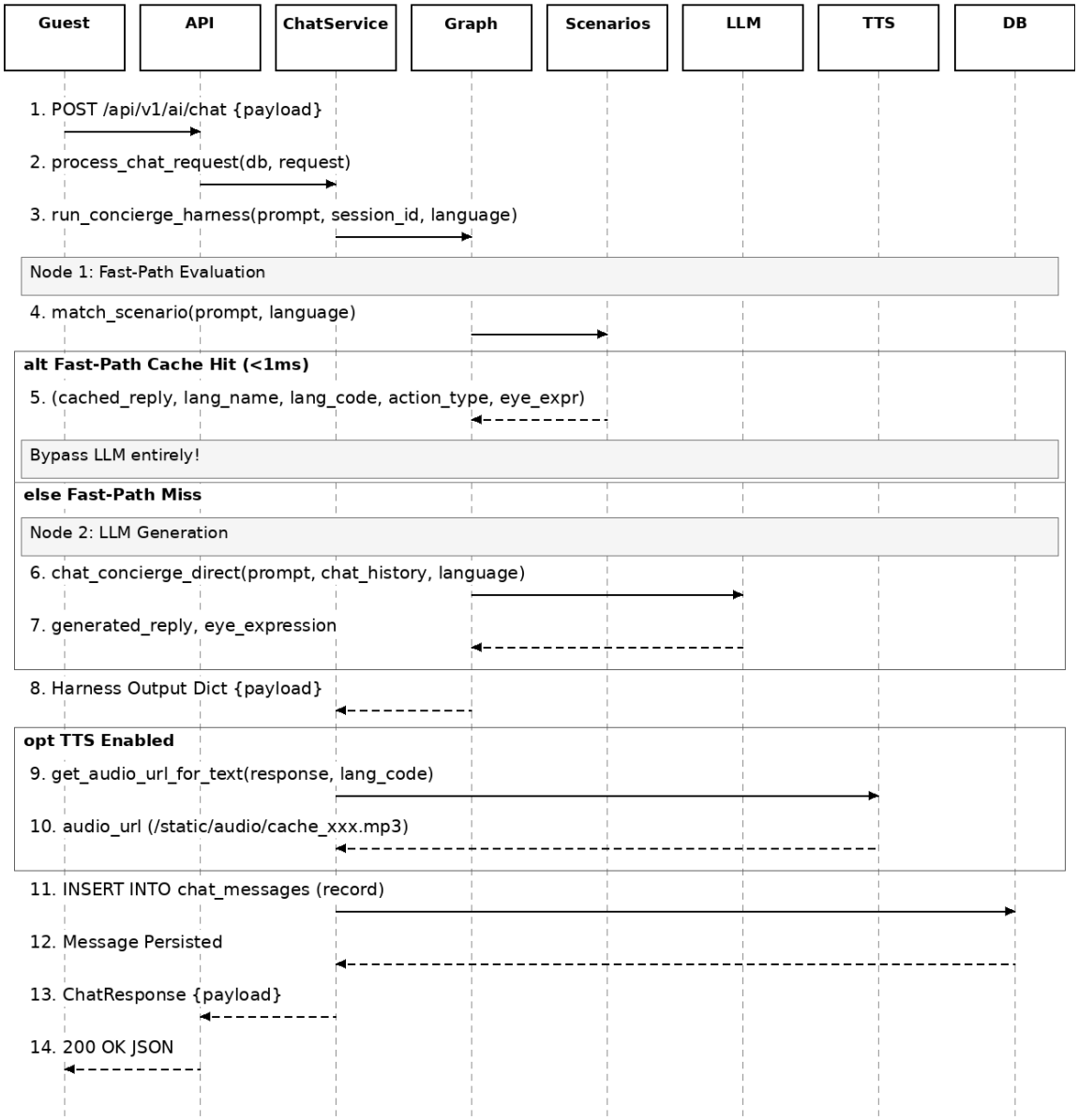


Figure 3.2-2. Process a guest question and return a spoken response.

Processing Rules and Implementation Limits

The illustrated sequence starts after text input is available; STTService handles WAV transcription separately. A scenario match bypasses the LLM. Conversation history is bounded to 10 turns according to the handoff. Provider order and deployment model require configuration confirmation. The handoff does not establish that ChromaDB retrieval is active in this conversational path, although Report 3 requires retrieval-grounded answers. No measured latency claim is made here.

3.3 Knowledge Base Management

Authorized content staff upload Markdown knowledge files or request reindexing. RAGRouter accesses the persistent concierge_kb collection through ChromaClientWrapper and uses ObsidianService to split source documents into chunks. Knowledge storage comprises the Markdown vault and ChromaDB rather than a PostgreSQL entity.

Report 3: 3.6.4 Knowledge Base and AI Scenarios; 3.4.2 Knowledge Retrieval and Answer Generation.

3.3.1 Class Diagram

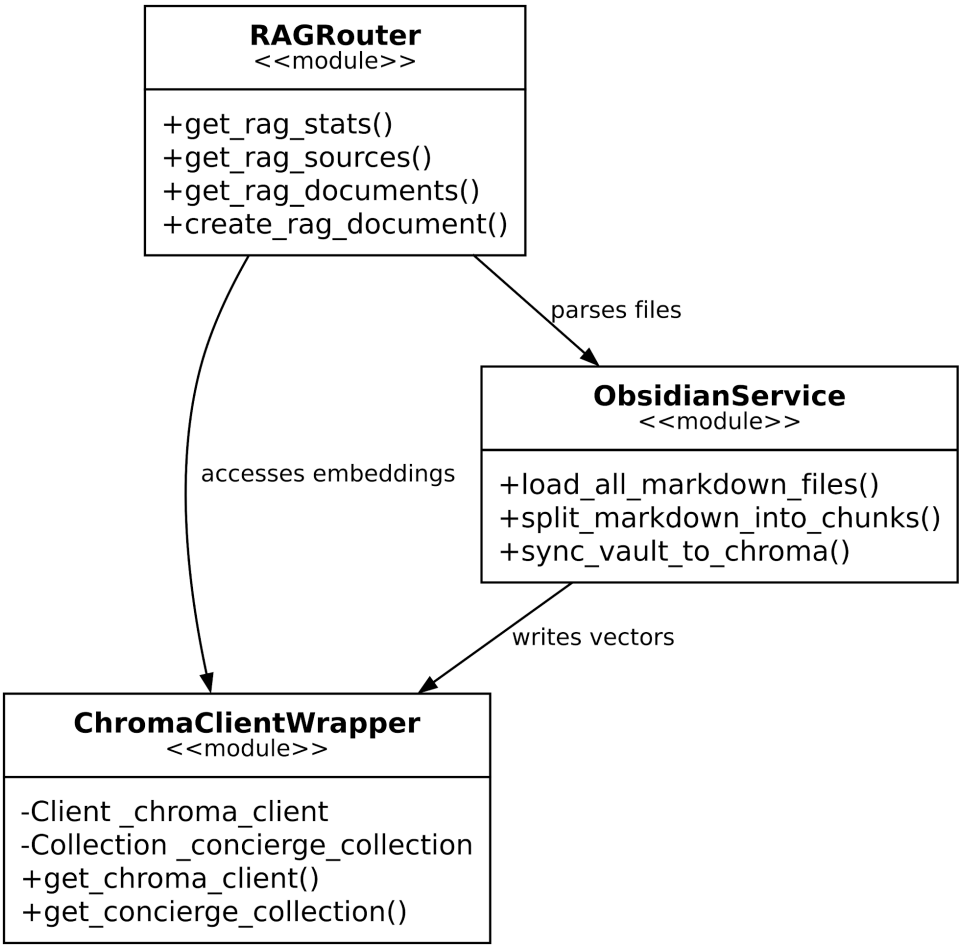


Figure 3.3-1. Knowledge Base Management: class and module relationships.

Class Specifications

No	Class / Module	Attribute / Method	Description
1	RAGRouter	get_rag_stats () -> RAGStatsResponse	Returns total documents, total sources, health percentage, category breakdown Source: app/api/v1/endpoints/rag.py:24
2	RAGRouter	get_rag_sources () -> RAGSourceFilesResponse	Scans .md files in Obsidian Vault with chunk counts and detected languages Source: app/api/v1/endpoints/rag.py:60
3	RAGRouter	reindex_all_sources () -> RAGActionResponse	Rebuilds ChromaDB collection from all .md files in Obsidian Vault Source: app/api/v1/endpoints/rag.py:284
4	RAGRouter	upload_knowledge_file async (file: UploadFile, category: str) -> RAGFileUploadResponse	Saves .md file to vault and immediately synchronizes to ChromaDB Source: app/api/v1/endpoints/rag.py:230
5	ChromaClientWrapper	get_concierge_collection (collection_name: str = "concierge_kb") -> Collection	Singleton accessor for persistent ChromaDB collection Source: app/services/rag/chroma.py:32
6	ObsidianService	split_markdown_into_chunks (raw_content: str, source_name: str) -> List[Dict]	Splits Markdown on headings into document chunks. Source: app/services/rag/obsidian_service.py:45

3.3.2 Upload and index a Markdown knowledge document

Participants: Admin = Hotel Admin; API = RAG Router; Disk = Obsidian Vault Storage; Service = ObsidianService; Chroma = ChromaDB Collection.

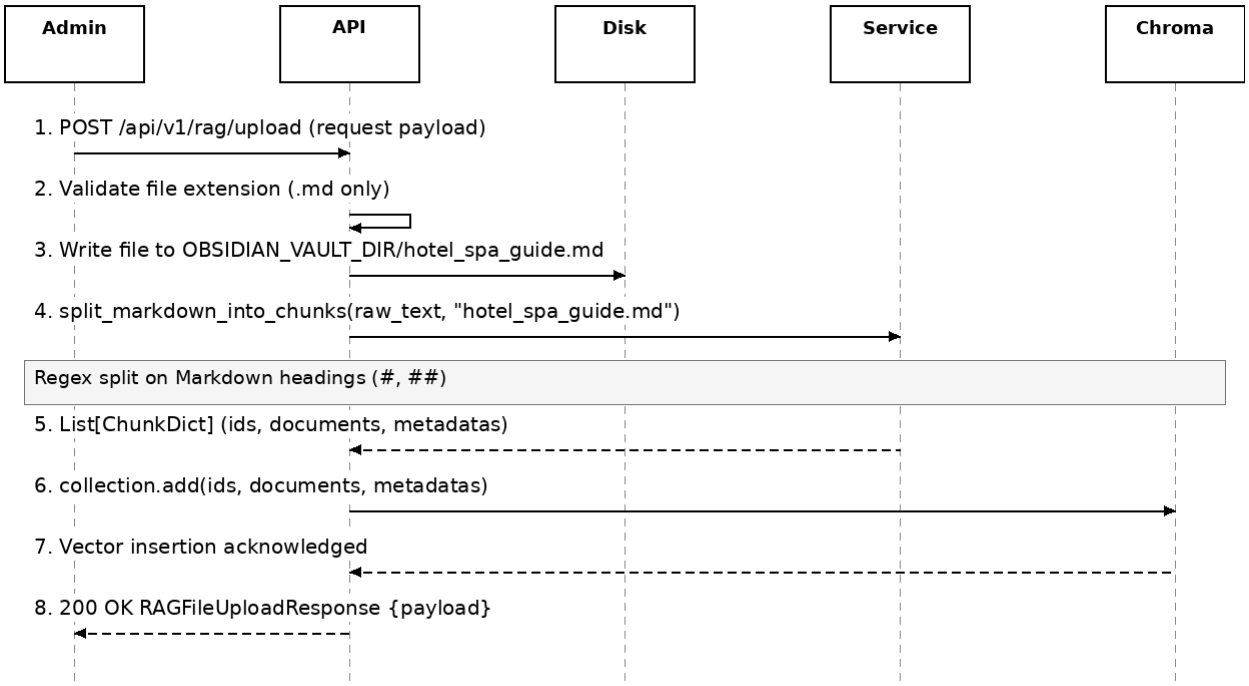


Figure 3.3-2. Upload and index a Markdown knowledge document.

Processing Rules and Implementation Limits

Only Markdown uploads are accepted; invalid extensions return HTTP 400. Headings define chunk boundaries and sections shorter than 30 characters are discarded. The handoff describes filename/frontmatter/content-based language detection. Upload/indexing is documented separately from conversational retrieval because the handoff explicitly reports a bypass of the full RAG generation path.

3.4 Robot Mapping and Navigation

MapRouter coordinates occupancy-grid access, saved maps, waypoint and zone persistence, and the ROS 2 runtime bridge. RPLidarService exposes scan/grid data; Ros2RuntimeService starts or stops the runtime and submits navigation or teleoperation commands. MapRepository stores Endpoint and Zone records.

Report 3: 3.3.3 Hotel Facilities and Floor Map; 3.6.3 Robot Control, Map and Navigation; 3.7 Robot Monitoring.

3.4.1 Class Diagram

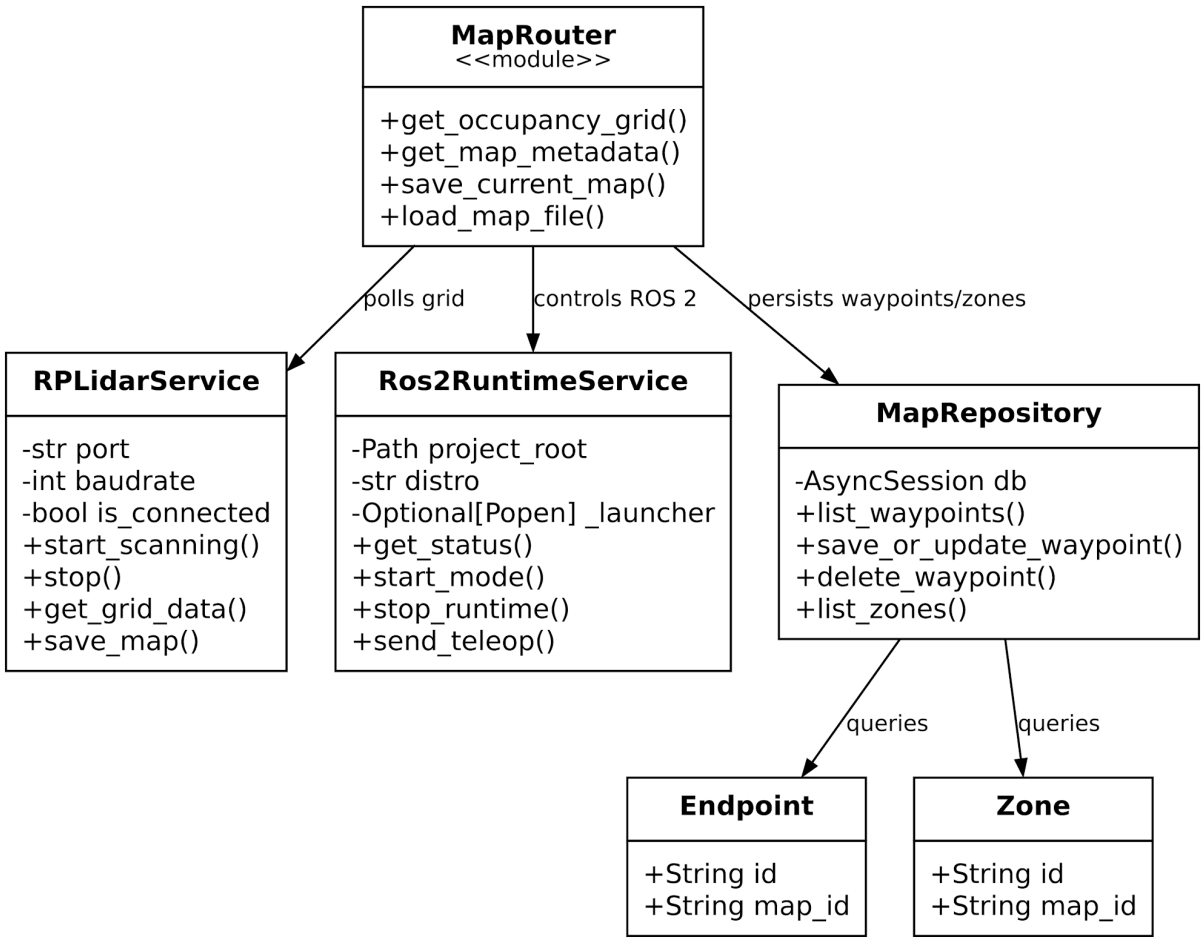


Figure 3.4-1. Robot Mapping and Navigation: class and module relationships.

Class Specifications

No	Class / Module	Attribute / Method	Description
1	RPLidarService	get_grid_data () -> List[int]	Returns 1D array of 40,000 cells (200x200 grid) Source: app/services/hardware/rplidar_service.py:180
2	RPLidarService	save_map (map_id: str) -> dict	Exports static .png and .json map files Source: app/services/hardware/rplidar_service.py:310
3	Ros2RuntimeService	send_navigation_goal (x: float, y: float, yaw: float) -> dict	Dispatches /navigate_to_pose goal to ROS 2 Nav2 Source: app/services/hardware/ros2_runtime_service.py:280
4	Ros2RuntimeService	cancel_navigation_goal () -> dict	Cancels active Nav2 navigation action Source: app/services/hardware/ros2_runtime_service.py:320
5	MapRepository	list_waypoints async () -> List[Endpoint]	Queries all navigation endpoints ordered by name Source: app/repositories/map_repository.py:30
6	MapRepository	save_or_update_zone async (...) -> Zone	Creates/updates Keep-out or Speed-limit zones Source: app/repositories/map_repository.py:95
7	MapRouter	navigate_to_waypoint async (req: NavigationRequest) -> NavigationResponse	Validates target point against Keep-Out zones then dispatches Nav2 goal Source: app/api/v1/endpoints/map.py:310

3.4.2 Validate a destination and dispatch a navigation goal

Participants: Staff = Staff / Job Dispatcher; API = Map Router; Repo = MapRepository; Ros2 = Ros2RuntimeService; Hardware = ROS 2 Nav2 Stack.

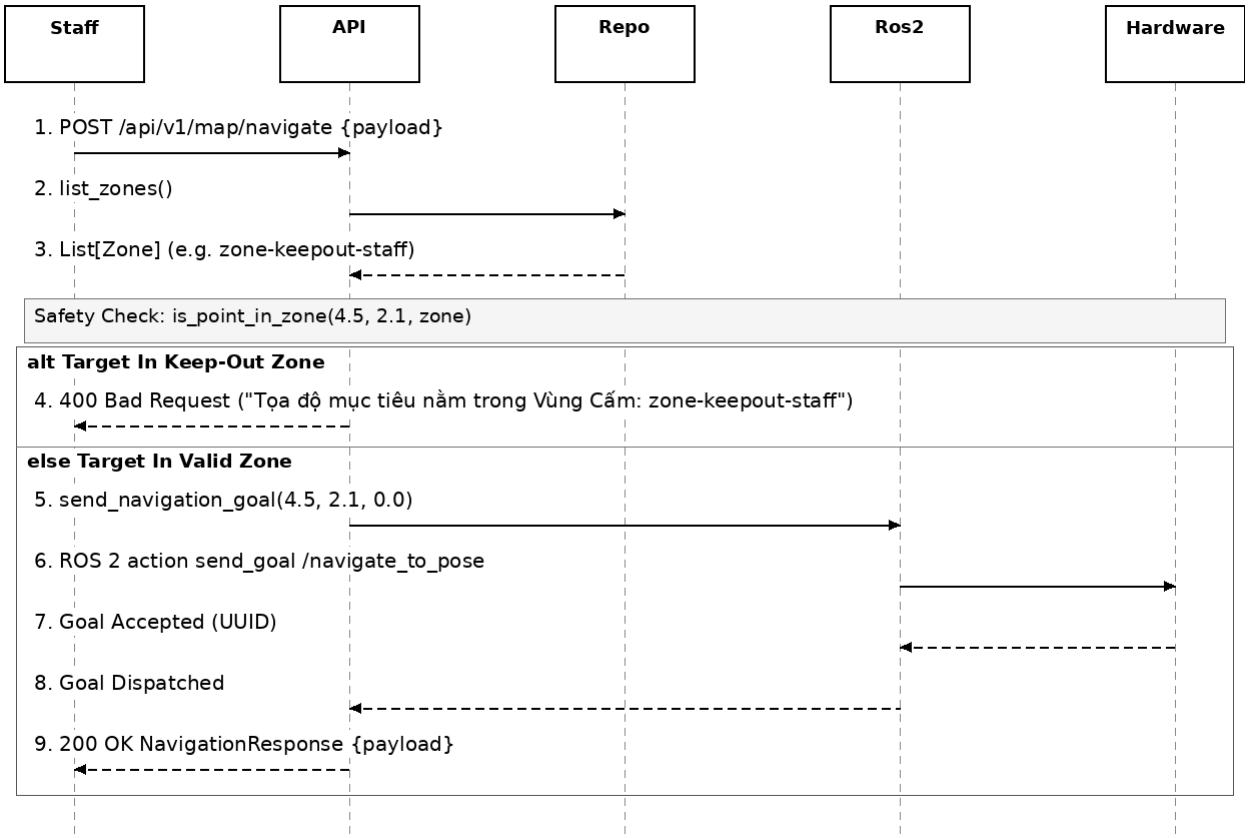


Figure 3.4-2. Validate a destination and dispatch a navigation goal.

Processing Rules and Implementation Limits

Before dispatch, the destination is checked against KEEP_OUT zone bounds. A target inside a prohibited zone returns HTTP 400. Goal acceptance means a navigation command was submitted; it does not establish arrival. Physical navigation requires an active ROS 2/Nav2 environment. Multi-floor elevator integration and physical obstacle-stop behavior are not established by this handoff.

3.5 Robot Workflows and Job Execution

WorkflowRouter manages reusable RobotWorkflow templates through WorkflowRepository. Executing a template creates a Job and ordered JobStep records for an assigned robot. JobEvent records represent execution events. Schedule associates a workflow with timing information, but a schedule record alone does not execute a job.

Report 3: 3.6.10 Robot Workflow Studio; 3.6.2 Operations Monitoring and Dispatch.

3.5.1 Class Diagram

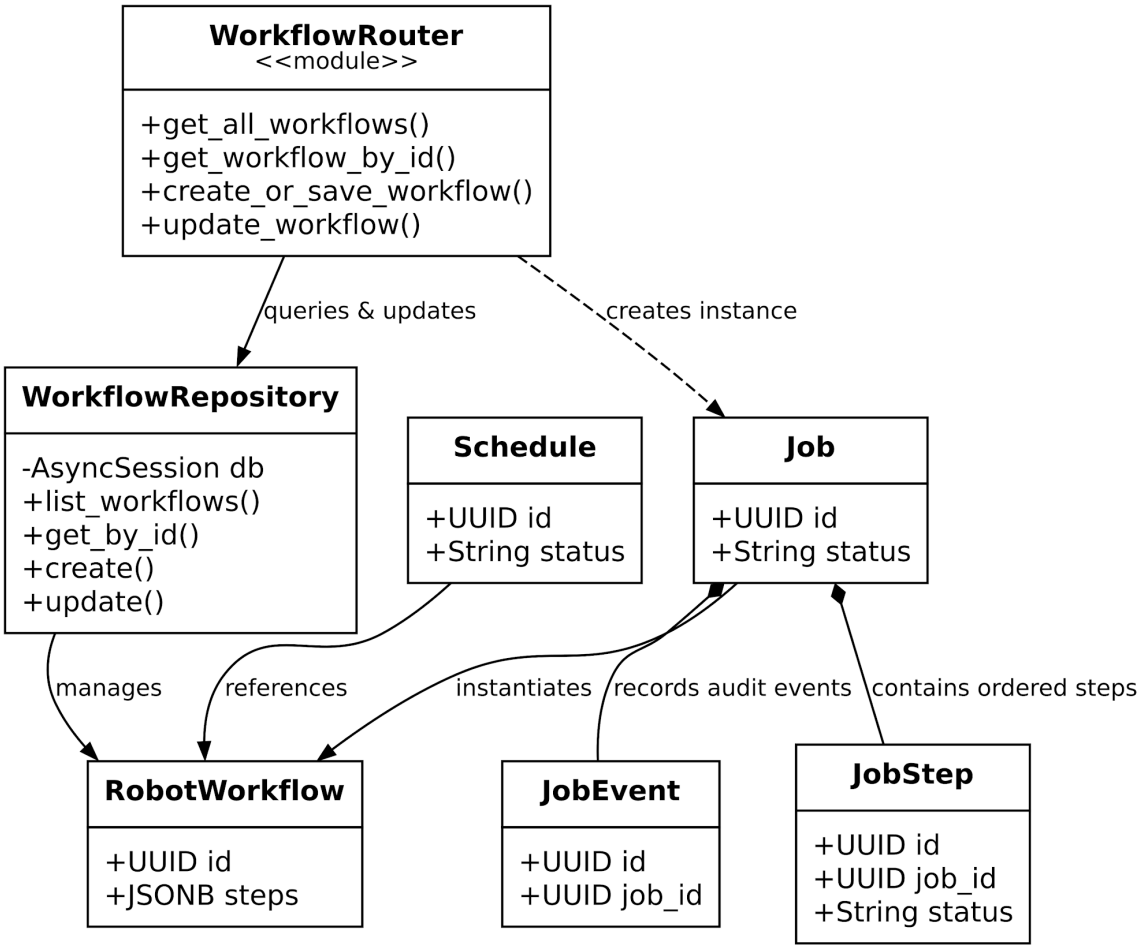


Figure 3.5-1. Robot Workflows and Job Execution: class and module relationships.

Class Specifications

No	Class / Module	Attribute / Method	Description
1	WorkflowRouter	create_or_save_workflow async (payload: WorkflowCreate, repo: WorkflowRepository) -> WorkflowResponse	Creates or saves custom robot workflow with JSON-encoded step sequence Source: app/api/v1/endpoints/workflows.py:48
2	WorkflowRouter	execute_workflow async (workflow_id: str, payload: WorkflowExecuteRequest, db: AsyncSession) -> Dict	Creates Job and JobStep records and initiates dispatch as described in the handoff; concrete dispatch integration requires confirmation. Source: app/api/v1/endpoints/workflows.py:112
3	WorkflowRepository	list_workflows async (active_only: bool = True) -> List[RobotWorkflow]	Fetches active workflow templates from database Source: app/repositories/workflow_repository.py:28
4	WorkflowRepository	seed_defaults async () -> List[RobotWorkflow]	Seeds default operational workflows (Room Escort, Food Delivery, Welcome Greeting) Source: app/repositories/workflow_repository.py:75
5	RobotWorkflow	steps Column(JSONB)	Array of action steps: [{"type": "NAVIGATE", "target": "ROOM_302"}, {"type": "SPEAK", "text": "Arrived"}] Source: app/models/workflow.py:19
6	Job	status Column(String)	Enum: PENDING, IN_PROGRESS, COMPLETED, FAILED, CANCELLED Source: app/models/job.py:25
7	JobStep	step_type Column(String)	Step primitive: NAVIGATE, SPEAK, WAIT_CONFIRMATION, CHARGE, INTERACT Source: app/models/job.py:68

3.5.2 Instantiate and dispatch a robot workflow

Participants: Staff = Hotel Staff; API = Workflows API; Repo = WorkflowRepository; DB = PostgreSQL; Manager = Robot Navigation/Serial Manager.

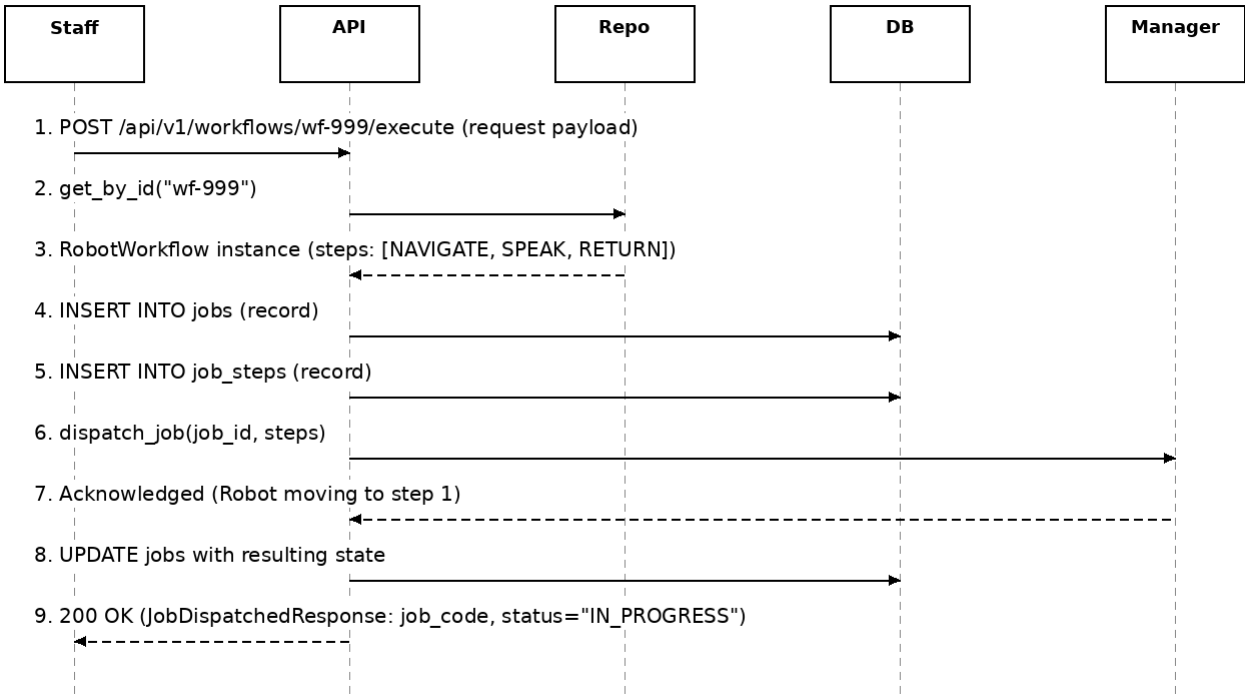


Figure 3.5-2. Instantiate and dispatch a robot workflow.

Processing Rules and Implementation Limits

The handoff reports PENDING, IN_PROGRESS, COMPLETED, FAILED and CANCELLED job states, a conflict response for a busy robot and failure events for aborted execution. Its overview uses different step/status names from its class specification; the specification names are used here pending source confirmation. The dispatch manager is shown as an interface-level participant because its concrete implementation and file are not identified. Recurring cron execution remains partial: no active scheduling worker is established.

3.6 Food and Beverage Room Service

Guests browse available menu items and submit an order for a room. RoomServiceRepository resolves the room and authoritative menu prices. The endpoint persists RoomServiceOrder and OrderItem records and notifies the kitchen. Order records retain item names, quantities and prices as the order snapshot.

Report 3: 3.3.4 Food Menu and Room Service Order; 3.5.4 Kitchen Menu Management; 3.5.6 Notifications.

3.6.1 Class Diagram

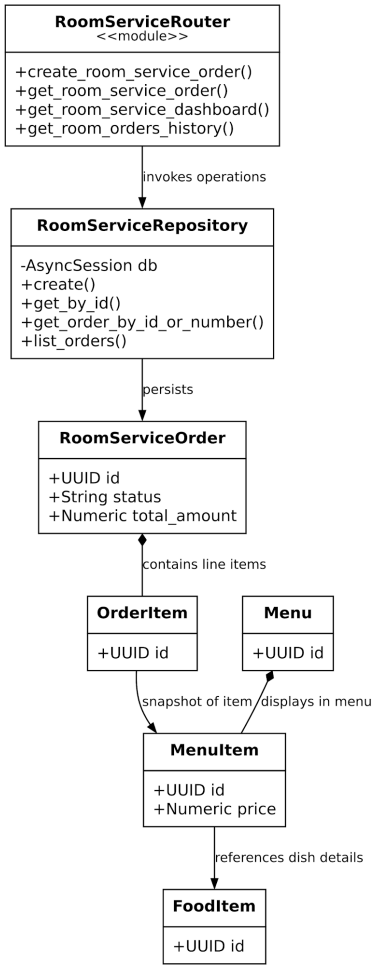


Figure 3.6-1. Food and Beverage Room Service: class and module relationships.

Class Specifications

No	Class / Module	Attribute / Method	Description
1	RoomServiceRouter	create_room_service_order async (order_in: RoomServiceOrderCreate, db: AsyncSession, repo: RoomServiceRepository) -> RoomServiceOrderResponse	Creates order, resolves room ID, computes line subtotals, broadcasts notification Source: app/api/v1/endpoints/operations/room_service.py:34
2	RoomServiceRouter	get_room_service_dashboard async (repo: RoomServiceRepository) -> Dict	Aggregates active orders, pending counts, revenue metrics Source: app/api/v1/endpoints/operations/room_service.py:112
3	RoomServiceRepository	lookup_food_and_menu_item async (item_id: str) -> Tuple[FoodItem, MenuItem]	Validates item existence and current pricing Source: app/repositories/room_service_repository.py:65
4	RoomServiceRepository	find_room async (room_number: str) -> Optional[Room]	Resolves room entity by number (e.g., "302") Source: app/repositories/room_service_repository.py:120
5	RoomServiceOrder	status Column(String)	Enum: PENDING, CONFIRMED, PREPARING, READY_FOR_DELIVERY, DELIVERING, DELIVERED, CANCELLED Source: app/models/room_service.py:22
6	OrderItem	subtotal Column(Numeric(10, 2))	Computed property: quantity * unit_price Source: app/models/room_service.py:72

3.6.2 Create a room-service order and notify the kitchen

Participants: Guest = Hotel Guest; API = Room Service Router; Repo = RoomServiceRepository; DB = PostgreSQL; WS = Notification Manager; Kitchen = Kitchen Staff.

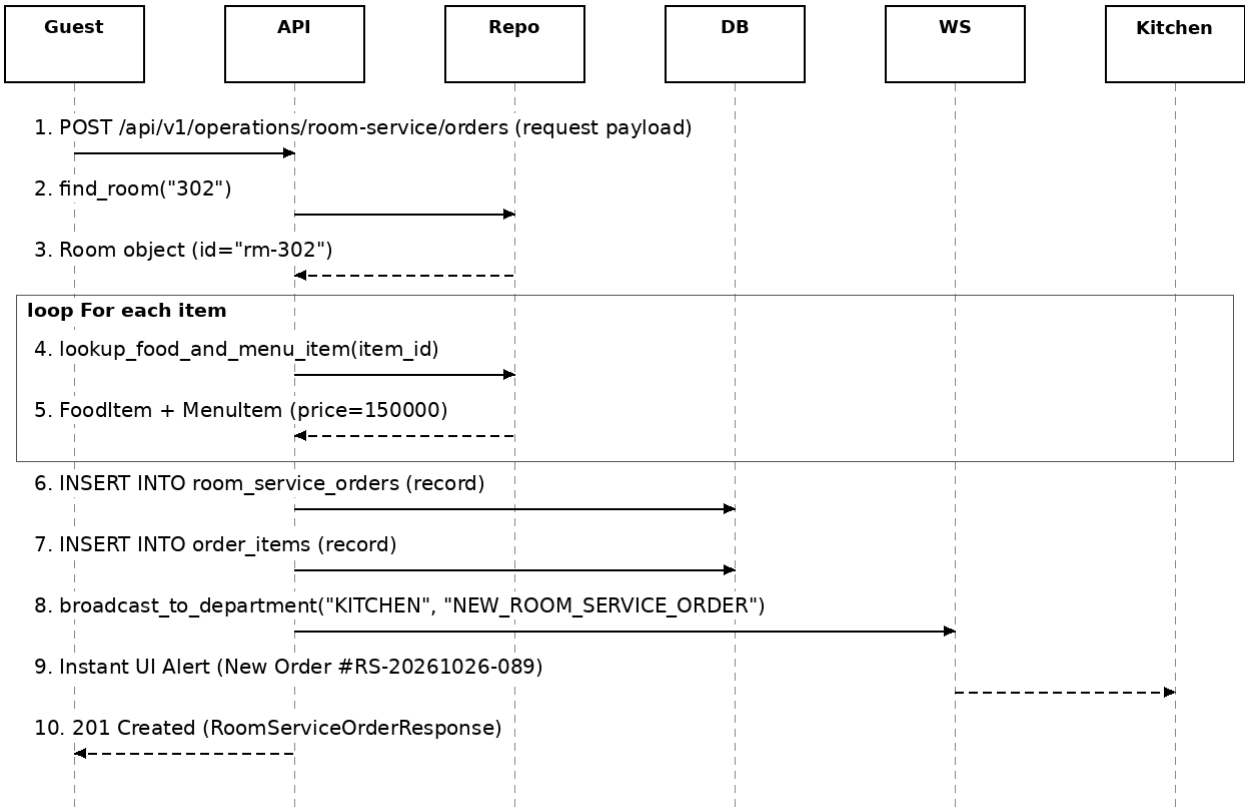


Figure 3.6-2. Create a room-service order and notify the kitchen.

Processing Rules and Implementation Limits

Totals are calculated on the backend from menu prices and item quantities. The handoff reports PENDING, CONFIRMED, PREPARING, READY_FOR_DELIVERY, DELIVERING, DELIVERED and CANCELLED order states. READY_FOR_DELIVERY is described as the delivery-job trigger, but the supplied sequence only establishes order creation and notification; dispatch ordering is not inferred. Payment-gateway integration is not active; settlement is described as room-charge/check-out.

3.7 Guest Service Requests and Room Verification

Reception, housekeeping, bell services, maintenance and transportation use department-specific endpoint modules with a shared SupportRequestRepository. Room verification resolves a room from a PIN or normalized NFC UID. A guest request becomes a departmental SupportRequest and is broadcast to the responsible staff channel.

Report 3: 3.3.5 Submit a Guest Service Request; 3.5.1 Department Dashboard; 3.5.2 Manage Guest Requests; 3.5.6 Notifications.

3.7.1 Class Diagram

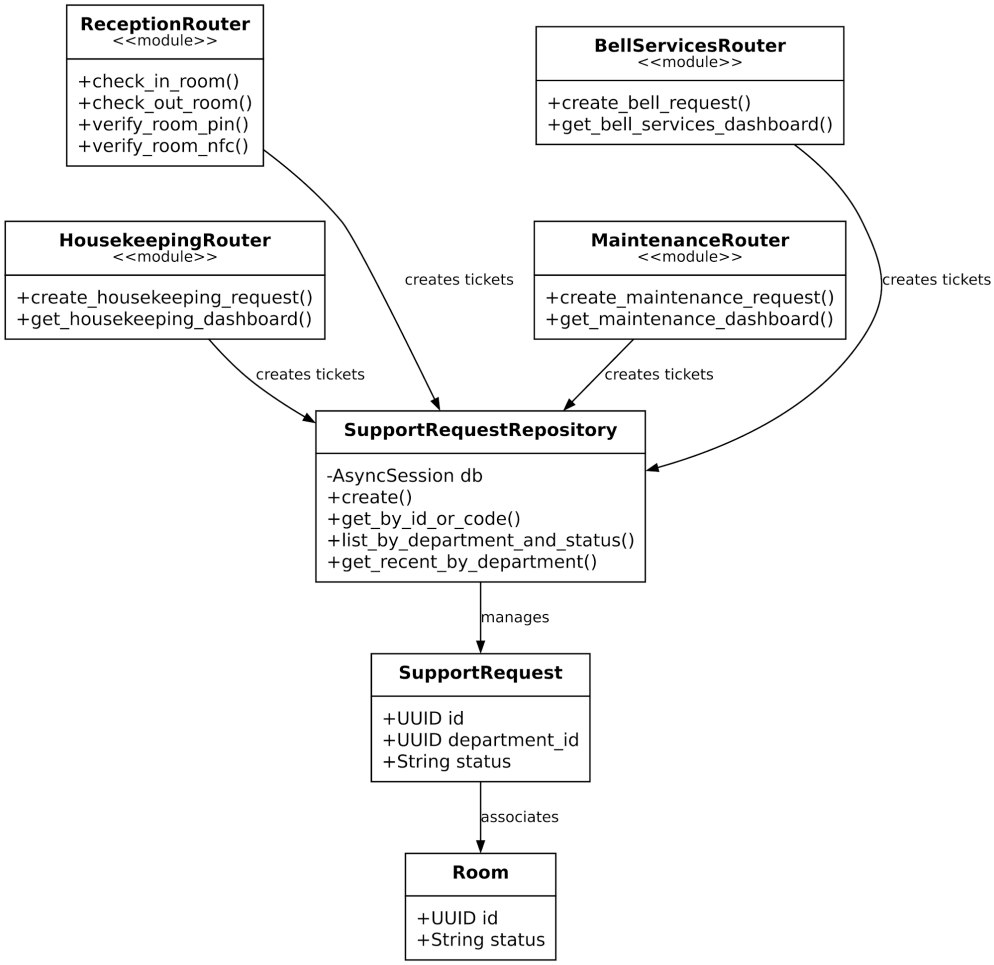


Figure 3.7-1. Guest Service Requests and Room Verification: class and module relationships.

Class Specifications

No	Class / Module	Attribute / Method	Description
1	ReceptionRouter	verify_room_pin async (payload: RoomPinVerifyRequest, db: AsyncSession) -> Dict	Validates the supplied PIN against rooms.pin_code; exact PIN-length constraint requires source confirmation. Source: app/api/v1/endpoints/operations/reception.py:55
2	ReceptionRouter	verify_room_nfc async (payload: RoomNfcVerifyRequest, db: AsyncSession) -> Dict	Normalizes and validates RFID/NFC card UID against rooms.nfc_card_uid Source: app/api/v1/endpoints/operations/reception.py:84
3	ReceptionRouter	check_in_room async (room_id: str, payload: RoomCheckInRequest, db: AsyncSession) -> Dict	Updates room status to OCCUPIED, sets guest credentials Source: app/api/v1/endpoints/operations/reception.py:122
4	SupportRequestRepository	create async (entity: SupportRequest) -> SupportRequest	Persists generic or department-specific service ticket Source: app/repositories/support_request_repository.py:34
5	SupportRequest	status Column(String)	Ticket state: PENDING, ACCEPTED, IN_PROGRESS, COMPLETED, CANCELLED Source: app/models/support.py:28
6	Room	status Column(String)	Room state: AVAILABLE, OCCUPIED, MAINTENANCE, DIRTY Source: app/models/hotel.py:32

3.7.2 Verify an NFC room card and submit a housekeeping request

Participants: Guest = Hotel Guest; Kiosk = Robot Kiosk NFC Reader / Screen; API = Operations API; Repo = SupportRequestRepository; DB = PostgreSQL; WS = WebSocket Hub.

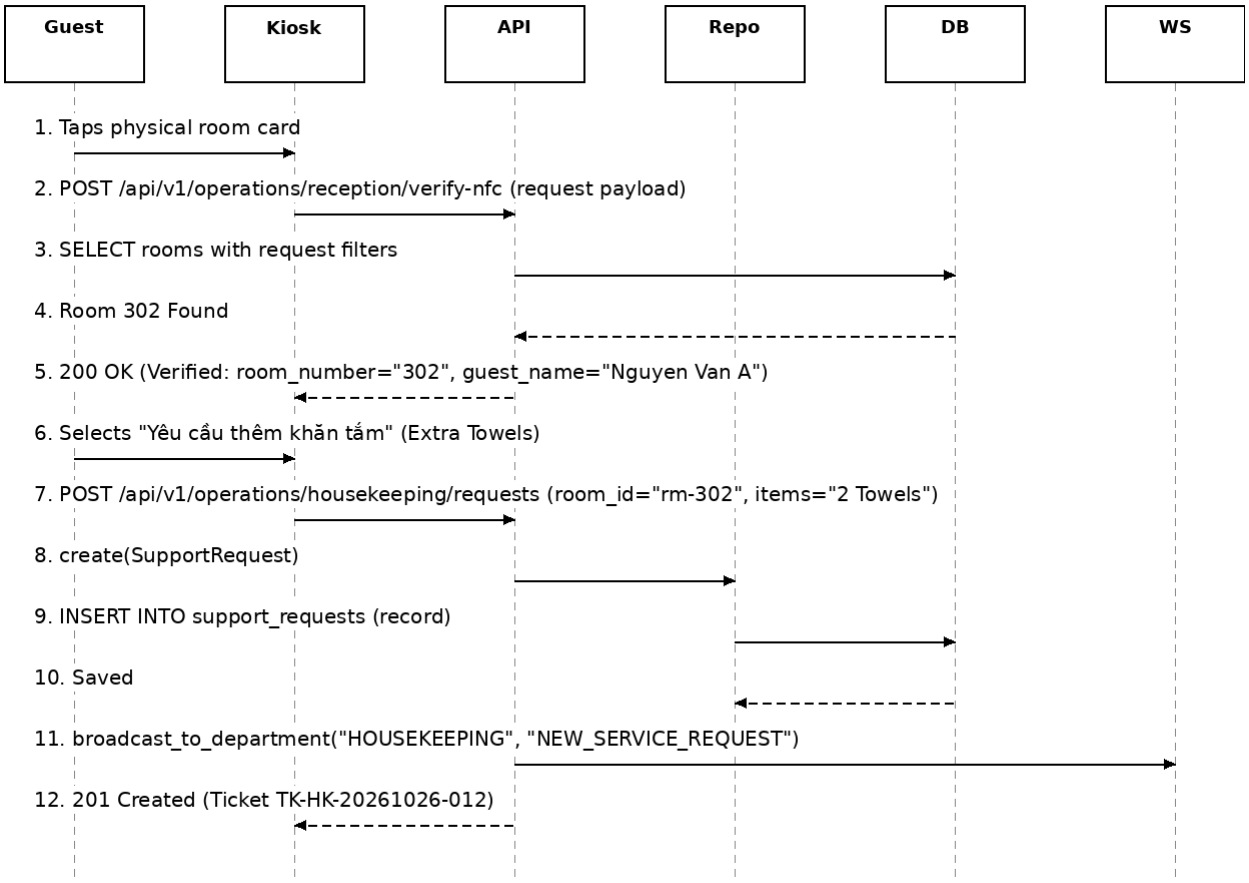


Figure 3.7-2. Verify an NFC room card and submit a housekeeping request.

Processing Rules and Implementation Limits

The housekeeping sequence is the representative shared ticket-creation flow; other departments reuse the repository pattern with their own request data and routing. NFC normalization removes separators and normalizes letter case. Check-out invalidates the PIN. The handoff inconsistently specifies four versus four-to-six PIN digits, so the precise length is not asserted. Room verification is an implementation detail additional to Report 3 actor descriptions. External PMS synchronization is not established.

3.8 Human Video Support and Recordings

ConciergeRouter and VideoCallManager relay WebRTC signaling between guest and staff clients. SDP offers, answers and ICE candidates pass through the signaling room; media flows directly between peers when connectivity permits. A completed recording is uploaded through CloudinaryService and the session metadata is updated.

Report 3: 3.3.6 Request Human Support; 3.5.7 Live Call Center and Recordings.

3.8.1 Class Diagram

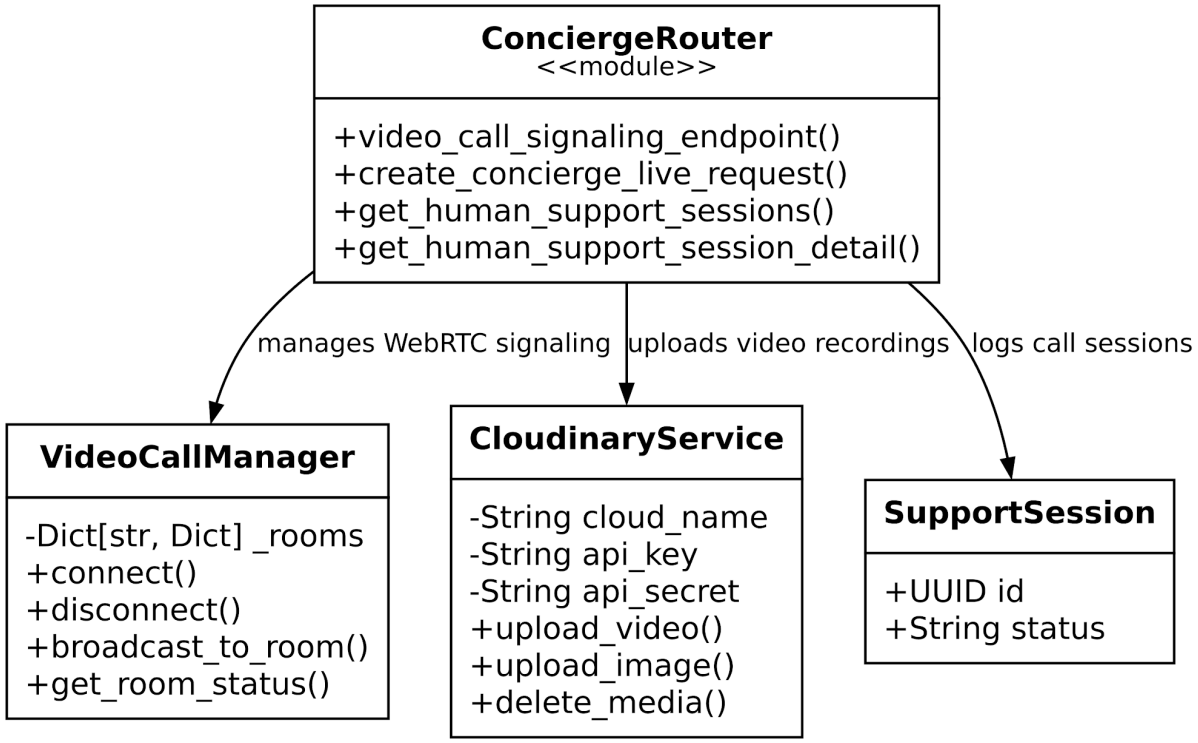


Figure 3.8-1. Human Video Support and Recordings: class and module relationships.

Class Specifications

No	Class / Module	Attribute / Method	Description
1	ConciergeRouter	video_call_signaling_endpoint async (ws: WebSocket, session_id: str, role: str, name: str)	WebSocket endpoint handling SDP offer/answer, ICE candidates, room state Source: app/api/v1/endpoints/operations/concierge.py:320
2	ConciergeRouter	upload_call_recording async (session_id: str, video_file: UploadFile, duration: int) -> RecordingUploadResponse	Uploads .webm/.mp4 call capture to Cloudinary and persists URL Source: app/api/v1/endpoints/operations/concierge.py:240
3	VideoCallManager	broadcast_to_room async (session_id: str, message: Dict, exclude: WebSocket)	Forwards WebRTC signaling packets to the peer connection Source: app/services/video_call_manager.py:82
4	CloudinaryService	upload_video (file_bytes: bytes, folder: str, public_id: str) -> Dict	Direct streaming upload to Cloudinary CDN with resource_type="video" Source: app/services/cloudinary_service.py:45
5	SupportSession	recording_url Column(String)	Public Cloudinary HTTPS URL for archived call playback Source: app/models/support.py:88

3.8.2 Establish a video call and archive its recording

Participants: Guest = Hotel Guest; KioskWS = WebSocket Client; Server = Concierge WebSocket Endpoint; Manager = VideoCallManager; StaffWS = WebSocket Client; Staff = Front Desk Staff; Cloudinary = cloud storage service; DB = session persistence.

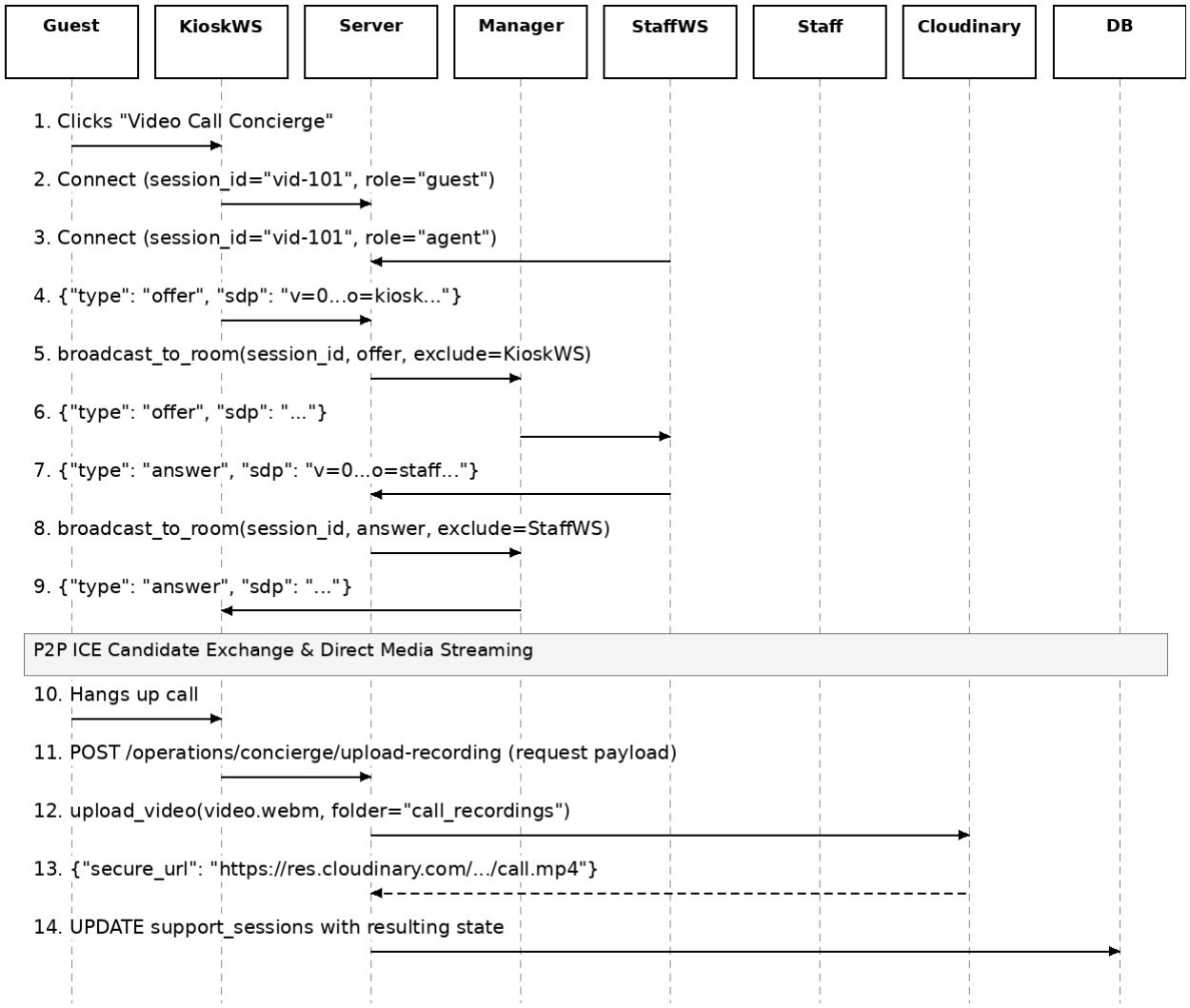


Figure 3.8-2. Establish a video call and archive its recording.

Processing Rules and Implementation Limits

The handoff specifies guest/agent signaling roles and one peer of each role per room. Recording upload uses HTTP, separately from the signaling WebSocket. A local recording fallback is described for cloud upload failure. Only STUN configuration is established; TURN deployment is not established. The handoff calls the model SupportSession/support_sessions while the existing database diagram uses human_support_sessions; this naming mismatch requires source/schema reconciliation and is not silently resolved here.

3.9 Guest Ratings and Feedback

AIRouter accepts a guest evaluation and delegates persistence to FeedbackService. Feedback records contain the rating, category, comment and optional conversation association. The shared service also exposes feedback retrieval and basic sentiment analysis.

Report 3: 3.3.7 Rate the Robot's Assistance; 3.6.1 Admin Dashboard and Statistics.

3.9.1 Class Diagram

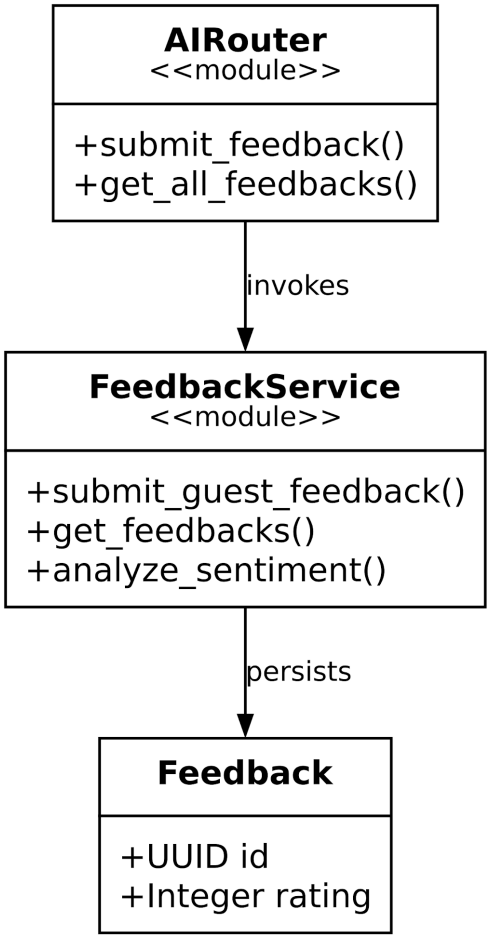


Figure 3.9-1. Guest Ratings and Feedback: class and module relationships.

Class Specifications

No	Class / Module	Attribute / Method	Description
1	AIRouter	submit_feedback async (payload: FeedbackCreate, db: AsyncSession) -> FeedbackResponse	Validates rating bounds (1-5) and stores guest evaluation Source: app/api/v1/endpoints/ai.py:215
2	FeedbackService	submit_guest_feedback async (db: AsyncSession, payload: FeedbackCreate) -> Feedback	Inserts entity and dispatches low-rating alert if rating <= 2 Source: app/services/ai/feedback_service.py:18
3	Feedback	rating Column(Integer, nullable=False)	Star rating between 1 and 5 Source: app/models/feedback.py:16
4	Feedback	category Column(String)	Category: CONCIERGE_AI, ROOM_SERVICE, CLEANLINESS, GENERAL Source: app/models/feedback.py:17

3.9.2 Submit and persist a guest rating

Participants: Guest = Hotel Guest; API = AI Endpoint; Service = FeedbackService; DB = PostgreSQL.

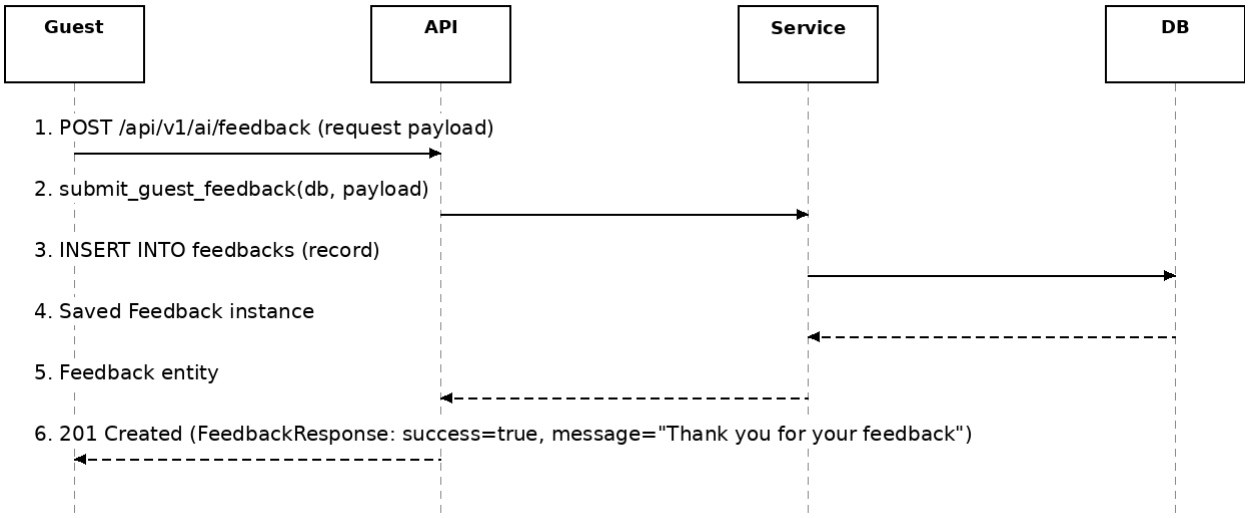


Figure 3.9-2. Submit and persist a guest rating.

Processing Rules and Implementation Limits

Ratings are constrained to 1 through 5. The chat-session association is optional. The handoff describes a low-rating alert for ratings at or below 2, but its supplied sequence only establishes persistence and acknowledgement; no additional alert-delivery order is inferred. Sentiment analysis is keyword-based and does not establish the facial-expression/attitude analysis described in Report 3.

3.10 Audit Logs and Advertisement Display

LogsRouter provides audit queries and correlated event traces through AuditLogRepository and LogEvent records. AdvertisementsRouter persists uploaded promotional media and returns active advertisements for idle kiosk playback. These are separate responsibilities grouped here to match the supplied handoff; the sequence below covers advertisement upload and playback.

Report 3: 3.6.8 Logs and Audit Trail; 3.6.5 Robot Advertisement Management; 3.3.8 Idle Mode Advertisement Display.

3.10.1 Class Diagram

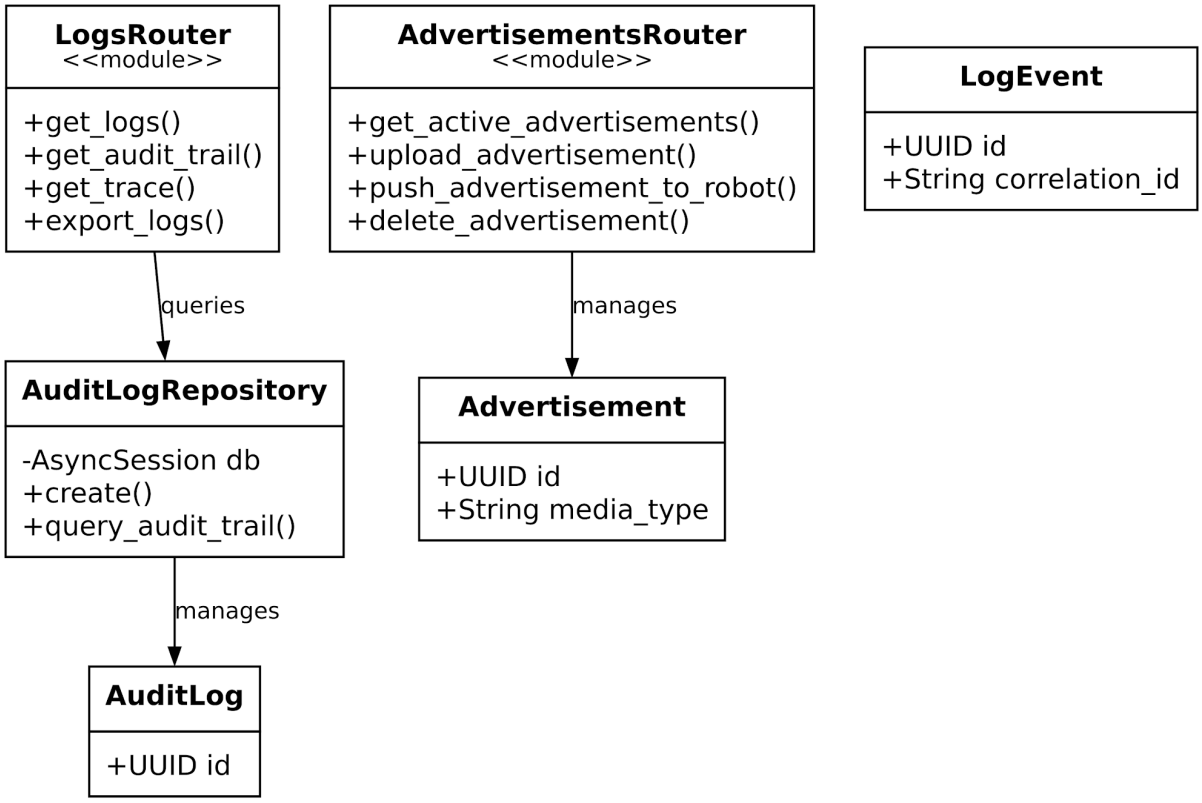


Figure 3.10-1. Audit Logs and Advertisement Display: class and module relationships.

Class Specifications

No	Class / Module	Attribute / Method	Description
1	LogsRouter	get_audit_trail async (page: int, limit: int, action: str, repo: AuditLogRepository) -> Dict	Paginated query of administrative actions with actor metadata Source: app/api/v1/endpoints/logs.py:65
2	LogsRouter	get_trace async (correlation_id: str, db: AsyncSession) -> List[Dict]	Reconstructs chronological event trace across all services by correlation ID Source: app/api/v1/endpoints/logs.py:120
3	AdvertisementsRouter	get_active_advertisements async (db: AsyncSession) -> List[Advertisement]	Fetches active campaigns ordered by priority for kiosk slideshow Source: app/api/v1/endpoints/advertisements.py:35
4	AdvertisementsRouter	upload_advertisement async (file: UploadFile, title: str, db: AsyncSession) -> Advertisement	Direct upload to Cloudinary CDN and persistence of media record Source: app/api/v1/endpoints/advertisements.py:82
5	AuditLog	details Column(JSONB)	Snapshot of changed fields, before/after states, and HTTP request body Source: app/models/logging.py:48
6	Advertisement	media_type Column(String)	Enum: IMAGE, VIDEO Source: app/models/advertisement.py:22

3.10.2 Upload promotional media and load the idle playlist

Participants: Admin = Marketing Staff; API = Ads API; Cloud = Cloudinary Service; DB = PostgreSQL; Robot = Concierge Robot.

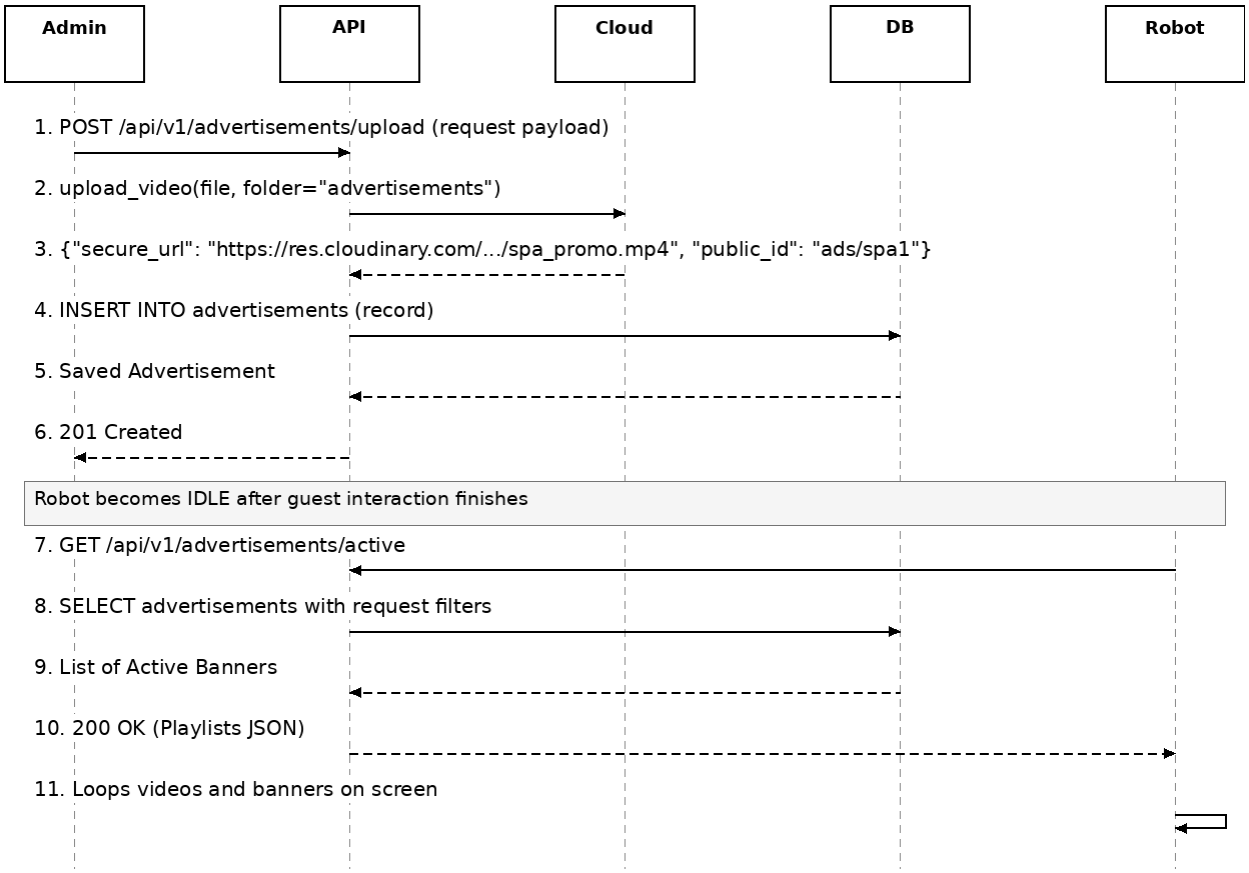


Figure 3.10-2. Upload promotional media and load the idle playlist.

Processing Rules and Implementation Limits

CorrelationIdMiddleware assigns or propagates the request correlation ID for trace queries. Audit records are described as append-only through the API, rather than as a database-level immutability guarantee. The advertisement list is filtered by active status and ordered by priority. Impression/gaze analytics are stubbed. The handoff provides no separate audit-writing sequence, so an exact call order for that operation is not invented.